

**PEMODELAN MACHINE LEARNING UNTUK
PRIORITISASI DAN ESTIMASI KEBUTUHAN
FASILITAS KESEHATAN KOTA BANDUNG**

TESIS

Oleh

**Dartika Anie Marian
23524319**



**PROGRAM STUDI MAGISTER INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

Mei 2026

LEMBAR PENGESAHAN

Pemodelan Machine Learning untuk Prioritisasi dan Estimasi Kebutuhan Fasilitas Kesehatan Kota Bandung

TESIS

Oleh

Dartika Anie Marian
23524319

Program Studi Magister Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Tesis ini telah disetujui dan disahkan
di Bandung, pada tanggal 23 Mei 2026

Pembimbing

Dr.Ir. Arry Akhmad Arman, MT.

NIP. 196504141991021001

PERNYATAAN ORISINALITAS

Saya yang bertanda tangan di bawah ini menyatakan dengan sesungguhnya bahwa:

1. Tesis ini adalah hasil karya saya sendiri yang dibuat dengan sebenar-benarnya sesuai dengan kaidah ilmiah dan etika akademik.
2. Semua sumber rujukan dan data yang saya gunakan dalam penyusunan tesis ini, baik yang berupa kutipan langsung maupun tidak langsung, telah saya cantumkan sumbernya dengan baik dan benar sesuai dengan ketentuan penulisan tesis.
3. Isi tesis ini belum pernah diajukan pada program pendidikan di institusi mana pun.

Apabila di kemudian hari terbukti bahwa dokumen tesis ini melanggar hal-hal di atas, saya bersedia menerima sanksi sesuai dengan peraturan yang berlaku di Institut Teknologi Bandung.

Bandung, 23 Mei 2026

Dartika Anie Marian

NIM: 23534219

PERNYATAAN PENGGUNAAN AI

Saya yang bertanda tangan di bawah ini menyatakan bahwa dalam penulisan tesis ini, saya telah menggunakan alat bantu kecerdasan artifisial (AI) untuk beberapa aspek tertentu. Berikut adalah rincian penggunaan AI dalam penulisan tesis ini.

No.	Jenis	Alat Bantu	Tingkat	Tempat
1	Memeriksa ejaan dan tata bahasa	Grammarly	Tinggi	Semua bab
2	Pembuatan teks	Microsoft Copilot	Sedang	Bab II dan Subbab III.2 hal. 34
3	Pembuatan grafik, gambar, atau ilustrasi	Tidak ada	Tidak ada	Tidak ada
4	Pencarian informasi atau referensi	...	...	...
5	Penyusunan bahan diskusi	...	...	...
6	Penyusunan kode program	...	...	...
7	Penyusunan formula atau perhitungan matematis	...	...	...
8	Penyusunan konsep desain atau algoritma	...	...	...
9	Penyusunan analisis data	...	...	...
10	Penyusunan kesimpulan atau rekomendasi	...	...	...

Bandung, 23 Mei 2026

Dartika Anie Marian

NIM: 23524319

ABSTRAK

Implementasi Sistem Smart Scale Berbasis IoT dengan Arsitektur Desentralisasi untuk Optimalisasi Rantai Pasok Jeruk di Tingkat Lapak

oleh

Dartika Anie Marian

23524319

Kata kunci:

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, saya dapat menyelesaikan Tesis yang berjudul "Pemodelan Machine Learning untuk Prioritisasi dan Estimasi Kebutuhan Fasilitas Kesehatan Kota Bandung". Tesis ini disusun sebagai salah satu syarat untuk menyelesaikan program Magister Informatika di Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

Bandung, 20 Mei 2026

Penulis

Dartika Anie Marian

DAFTAR ISI

DAFTAR GAMBAR

DAFTAR TABEL

DAFTAR SINGKATAN

Singkatan	Deskripsi	Pemakaian pertama kali
AI	<i>Artificial Intelligence</i>	1
CNN	<i>Convolutional Neural Network</i>	2
GPU	<i>Graphics Processing Unit</i>	14

BAB I

PENDAHULUAN

I.1 Latar Belakang

Perkembangan *Large Language Model* (LLM) dalam beberapa tahun terakhir telah mendorong transformasi signifikan pada berbagai aplikasi kecerdasan buatan, termasuk natural language understanding, text generation, conversational agents, machine translation, serta code generation (**kumar2024; qin2025**). Model-model seperti GPT-4 dan LLaMA 2 telah menunjukkan kemampuan luar biasa dalam memahami dan menghasilkan teks natural yang menyerupai tulisan manusia. Haque (2025) menegaskan bahwa LLM bukan hanya mengubah cara perangkat lunak dikembangkan, tetapi secara fundamental mendefinisikan ulang peran pengembang dalam siklus hidup rekayasa perangkat lunak (*software engineering*). Adopsi LLM di industri perangkat lunak juga meningkat secara signifikan, sebagaimana dikonfirmasi oleh Ronanki **and others** (2026) yang melakukan studi multi-kasus pada tiga organisasi yang menggunakan LLM dalam aktivitas rekayasa perangkat lunak dan mengidentifikasi perlunya praktik terbaik yang dapat ditindaklanjuti untuk adopsi LLM yang efisien dan bertanggung jawab. Meskipun demikian, di balik kemampuan generatif tersebut, terdapat permasalahan mendasar yang belum sepenuhnya terpecahkan yaitu kecenderungan LLM untuk menghasilkan informasi yang tampak meyakinkan namun faktanya tidak akurat atau bahkan sepenuhnya merupakan fabrikasi (Ji **and others** 2023).

Fenomena ini dikenal sebagai *hallucination* (halusinasi) dan menjadi salah satu hambatan paling kritis dalam penerapan LLM pada domain yang membutuhkan keakuratan tinggi. Ji **and others** (2023) mendefinisikan halusinasi sebagai kondisi di mana model menghasilkan keluaran yang tidak sesuai dengan fakta yang dapat diverifikasi, tidak konsisten dengan konteks yang diberikan, atau tidak memiliki dasar empiris yang dapat ditelusuri. Huang **and others** (2025) dalam survei komprehensifnya di *ACM Transactions on Information Systems* mengidentifikasi tiga lapisan akar penyebab halusinasi yang saling terkait: (1) kualitas dan kelengkapan data pelatihan yang tidak merata, (2) keterbatasan proses penyelarasan (*alignment*) model terhadap pengetahuan faktual, dan (3) mekanisme inferensi yang cenderung menghasilkan teks yang terdengar koheren tanpa jaminan akurasi faktual. Chang **and others** (2026) melengkapi analisis ini dengan mengidentifikasi tiga tantangan fundamental

dalam generasi teks berbasis LLM yang memperkuat kecenderungan halusinasi: (1) pemahaman instruksi yang tidak memadai pada konteks panjang dan percakapan multi-gilir, (2) penalaran intensi yang tidak konsisten, khususnya ketika instruksi mengandung informasi ambigu, dan (3) generasi dialog yang tidak andal (*unreliable dialogue generation*) yang menghasilkan konten tidak stabil dari prompt yang serupa.

Dalam konteks pemrograman dan rekayasa perangkat lunak, halusinasi LLM bermanifestasi dalam berbagai bentuk yang berpotensi menimbulkan dampak langsung terhadap kualitas sistem. Alomari **and others** (2026) dalam tinjauan literatur sistematis terhadap 49 studi penggunaan LLM untuk peningkatan kualitas kode menemukan bahwa kode yang dihasilkan atau direfaktorisasi oleh LLM tidak dapat diandalkan dan masih menghadapi tantangan besar dalam hal optimasi dan keandalan. Sebuah temuan yang mengonfirmasi bahwa pengetahuan parametrik LLM saja tidak cukup untuk menjamin keakuratan di domain teknis yang terus berkembang. Silva, Samhi **and Khomh** (2025) memperkuat bukti ini melalui studi empiris yang membandingkan jawaban ChatGPT dan LLaMA terhadap pertanyaan Stack Overflow. Hasil penelitian Silva, Samhi **and Khomh** (2025) menunjukkan bahwa meskipun LLM menantang keahlian manusia, LLM tidak melampaui keahlian komunitas manusia untuk sejumlah domain tertentu. Yang lebih signifikan, Silva, Samhi **and Khomh** (2025) mendokumentasikan penurunan nyata aktivitas *posting* pengguna di Stack Overflow sejak hadirnya ChatGPT. Pengguna mulai mengandalkan LLM meskipun keandalannya belum terjamin penuh, sehingga berpotensi memperparah penyebaran informasi teknis yang tidak akurat.

Stack Overflow yang merupakan platform utama pengembang untuk pertanyaan seputar pemrograman dan pengembangan perangkat lunak, kini berada di persimpangan antara ancaman dan peluang akibat kehadiran LLM. Di satu sisi, aktivitas penggunaannya menurun (Silva, Samhi **and Khomh** 2025). Di sisi lain, komunitas Stack Overflow menyimpan lebih dari 58 juta pasang pertanyaan dan jawaban yang telah melewati mekanisme kurasi ketat berupa sistem *upvote/downvote*, penanda jawaban yang diterima (*accepted answer*), dan sistem reputasi kontributor (Kabir **and others** 2024). Sinyal kepercayaan komunitas ini menjadikan Stack Overflow bukan sekadar repositori teks, melainkan basis pengetahuan tervalidasi secara sosial yang potensial digunakan untuk memberikan landasan pengetahuan bagi jawaban LLM berdasarkan fakta yang telah diverifikasi oleh praktisi domain. Nashid, Sintaha **and Mesbah** (2023) menunjukkan bahwa pemanfaatan konten Stack Overflow sebagai sumber pengambilan konteks secara signifikan meningkatkan kualitas ja-

waban LLM pada tugas-tugas terkait kode, menggarisbawahi potensi besar repositori ini.

Salah satu pendekatan yang paling menjanjikan untuk memitigasi halusinasi adalah *Retrieval-Augmented Generation* (RAG) (Lewis **and others** 2020). Huang **and** Huang (2026) dalam survei mereka di *ACM Computing Surveys* menjelaskan bahwa RAG menggabungkan teknik pengambilan informasi (*information retrieval*) dengan kemampuan generatif LLM untuk mengatasi keterbatasan pengetahuan statis model, sehingga memungkinkan integrasi informasi eksternal yang dinamis dan terkini pada saat inferensi. Cruz Cabello, Sales **and** Machado (2025) melalui tinjauan literatur sistematis pada domain deteksi anomali *log* sistem komputer menemukan bahwa RAG secara substansial meningkatkan akurasi dan interpretabilitas jawaban LLM tanpa memerlukan *fine-tuning* yang ekstensif, dan bahwa integrasi RAG dengan LLM menunjukkan kemampuan adaptasi yang kuat terhadap lingkungan operasional yang berubah. Temuan ini mengkonfirmasi potensi RAG sebagai strategi mitigasi halusinasi yang praktis dan skalabel di berbagai domain teknis.

Meskipun demikian, implementasi RAG konvensional yang menggunakan pencarian berbasis kemiripan vektor (*dense retrieval*) terhadap potongan-potongan teks yang tidak terstruktur memiliki keterbatasan mendasar yaitu pendekatan ini gagal memanfaatkan relasi semantik kompleks dan hubungan kontekstual antar konsep yang justru sangat penting dalam domain pemrograman (Huang **and** Huang 2026). Sebagai respons atas keterbatasan ini, pendekatan *Graph-based Retrieval-Augmented Generation* (GraphRAG) diusulkan untuk mengintegrasikan struktur *knowledge graph* ke dalam proses pengambilan konteks. Zhu **and others** (2026) dalam survei komprehensif mereka di *ACM Computing Surveys* mengidentifikasi tiga inovasi kunci GraphRAG dibandingkan RAG konvensional: (1) representasi pengetahuan berbasis graf yang secara eksplisit mengkodekan relasi antar entitas dan hierarki domain, (2) teknik pengambilan berbasis graf yang memungkinkan pengambilan konteks dengan kemampuan penalaran multi-hop, dan (3) algoritma integrasi pengetahuan yang sadar-struktur untuk menghasilkan teks yang lebih akurat dan koheren. Pan **and others** (2024) dalam menegaskan bahwa penggabungan LLM dengan *knowledge graph* membentuk paradigma baru yang secara signifikan meningkatkan kemampuan penalaran berbasis pengetahuan dan mengurangi fabrikasi faktual (*factual hallucination*).

Dari sisi evaluasi, Gu, Jiang, Shi **and others** (2026) dalam survei komprehensif mereka tentang *LLM-as-a-Judge* menegaskan bahwa mengukur keandalan LLM meru-

pakan tantangan signifikan yang membutuhkan desain dan standardisasi yang cermat. Mereka mengusulkan strategi untuk meningkatkan konsistensi evaluasi dan memitigasi bias, sebuah kerangka yang relevan untuk mengukur efektivitas reduksi halusinasi pada sistem berbasis GraphRAG dan menjadi prasyarat agar kontribusi sebuah *framework* mitigasi halusinasi dapat diklaim secara akademis.

Meskipun kemajuan di masing-masing bidang LLM, RAG, GraphRAG, dan penambahan pengetahuan komunitas cukup pesat, terdapat kesenjangan metodologis yang signifikan ketika keempat komponen ini dipertimbangkan secara terpadu. Tabel ?? merangkum metode yang telah digunakan oleh penelitian terkait beserta keterbatasannya terhadap tujuan penelitian ini.

Berdasarkan pemetaan kesenjangan pada Tabel ??, setidaknya terdapat tiga lapisan kesenjangan yang belum dijawab oleh penelitian sebelumnya. *Pertama*, penelitian yang secara langsung membandingkan LLM dengan Stack Overflow seperti Silva, Samhi **and** Khomh (2025) dan Kabir **and others** (2024), bersifat evaluatif. Mereka mengukur dan mendokumentasikan kesenjangan kualitas antara LLM dan keahlian komunitas, tetapi tidak mengusulkan mekanisme arsitektural untuk menjembatani kesenjangan tersebut. *Kedua*, penelitian yang menggunakan konten Stack Overflow sebagai sumber konteks untuk LLM, seperti Nashid, Sintaha **and** Mesbah (2023) hanya menerapkan RAG konvensional berbasis kemiripan vektor dan tidak memanfaatkan struktur relasional pengetahuan komunitas maupun sinyal kepercayaan komunitas (*upvote*, *accepted answer*) sebagai bobot kepercayaan dalam proses pengambilan konteks. *Ketiga*, meskipun pendekatan GraphRAG terbukti superior dibandingkan RAG konvensional pada berbagai domain (Zhu **and others** 2026), serta integrasi *knowledge graph* dengan LLM diyakini sebagai paradigma yang menjanjikan (Pan **and others** 2024), belum ada penelitian yang secara khusus membangun *knowledge graph* dari konten Stack Overflow yang mengkodekan sinyal kepercayaan komunitas untuk digunakan sebagai fondasi GraphRAG pada domain pemrograman. Sementara itu, tantangan halusinasi akibat keterbatasan pemahaman instruksi dan generasi dialog yang tidak andal (Chang **and others** 2026; Huang **and others** 2025) tetap menjadi masalah terbuka yang membutuhkan pendekatan grounding berbasis pengetahuan terverifikasi.

Kesenjangan-kesenjangan ini secara kolektif membuka peluang kontribusi yang signifikan: pengembangan sebuah *GraphRAG Framework* yang secara sistematis (1) membangun *knowledge graph* dari konten Stack Overflow yang mengkodekan entitas teknis, relasi semantik, *dan* sinyal kepercayaan komunitas sebagai bobot keper-

cayaan eksplisit; (2) merancang mekanisme *graph-guided retrieval* yang memanfaatkan struktur relasional tersebut untuk mengidentifikasi konteks yang lebih relevan dibandingkan RAG konvensional; (3) mengintegrasikan konteks hasil traversal ke dalam proses inferensi LLM; dan (4) mengukur efektivitas reduksi halusinasi secara kuantitatif menggunakan protokol evaluasi yang terstandarisasi.

I.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, kondisi ideal yang diharapkan adalah tersedianya mekanisme *grounding* berbasis pengetahuan komunitas yang tervalidasi secara sosial sehingga LLM dapat menghasilkan jawaban teknis yang akurat dan dapat dipertanggungjawabkan. Namun, kondisi aktual yang dijumpai menunjukkan kesenjangan metodologis yang signifikan: penelitian yang menggunakan konten Stack Overflow sebagai konteks LLM masih menggunakan RAG konvensional dan tidak memanfaatkan struktur relasional pengetahuan komunitas (Nashid, Sintaha **and** Mesbah 2023); pendekatan GraphRAG yang ada belum mengeksplorasi sumber pengetahuan bervalidasi sosial maupun domain pemrograman secara spesifik (Zhu **and others** 2026; Pan **and others** 2024); sementara itu, tantangan halusinasi dalam generasi teks LLM pada instruksi yang ambigu dan konteks panjang masih belum terpecahkan (Chang **and others** 2026; Huang **and others** 2025). Kesenjangan antara kondisi ideal dan kondisi aktual tersebut menjadi landasan permasalahan utama dalam penelitian ini, yang dirumuskan sebagai berikut.

Bagaimana merancang dan mengembangkan GraphRAG *Framework* berbasis pengetahuan komunitas Stack Overflow yang secara terukur mampu mereduksi halusinasi pada *Large Language Model* di domain pemrograman?

Untuk menjawab pertanyaan utama tersebut, penelitian ini mengajukan pertanyaan turunan sebagai berikut.

1. Bagaimana membangun *knowledge graph* yang merepresentasikan pengetahuan teknis komunitas Stack Overflow secara terstruktur, termasuk entitas (konsep, fungsi, pustaka, *error*), relasi antar entitas, serta sinyal kepercayaan komunitas (*upvote*, *accepted answer*, skor reputasi) sebagai bobot kepercayaan eksplisit pada simpul dan sisi graf?
2. Bagaimana merancang mekanisme *graph-guided retrieval* yang mampu mengidentifikasi subgraf relevan berdasarkan kueri pengguna, sedemikian sehingga konteks yang diekstraksi lebih kaya secara semantik dan relasional dibandingkan pengambilan berbasis kemiripan vektor pada RAG konvensional?

3. Bagaimana mengintegrasikan konteks hasil traversal *knowledge graph* Stack Overflow ke dalam proses inferensi LLM sehingga jawaban yang dihasilkan memiliki tingkat akurasi faktual yang lebih tinggi dibandingkan LLM tanpa mekanisme *grounding*?
4. Bagaimana merancang protokol evaluasi yang mampu mengukur dan memvalidasi efektivitas *framework* yang diusulkan dalam mereduksi halusinasi LLM menggunakan metrik kuantitatif yang terstandarisasi, serta membandingkan kinerja *framework* terhadap dua kondisi *baseline*: LLM murni dan RAG konvensional?

I.3 Tujuan

Tujuan penelitian ini dibagi menjadi tujuan umum dan tujuan khusus. Tujuan umum penelitian ini adalah mengembangkan *GraphRAG Framework* berbasis pengetahuan komunitas Stack Overflow sebagai artefak penelitian desain yang mampu mereduksi halusinasi LLM pada domain pemrograman secara terukur, sekaligus menghasilkan kontribusi pengetahuan berupa prinsip-prinsip desain yang dapat digunakan kembali oleh penelitian serupa di domain teknis lainnya.

Adapun tujuan khusus dari penelitian ini adalah sebagai berikut.

1. Membangun *knowledge graph* dari data Stack Overflow yang merepresentasikan entitas teknis, relasi semantik antar entitas, dan sinyal kepercayaan komunitas sebagai bobot pada simpul dan sisi graf.
2. Merancang dan mengimplementasikan modul *graph-guided retrieval* yang memanfaatkan traversal *knowledge graph* untuk mengekstraksi konteks yang relevan secara semantik dan relasional berdasarkan kueri pengguna, melampaui keterbatasan RAG konvensional berbasis kemiripan vektor.
3. Mengembangkan modul *grounded generation* yang mengintegrasikan konteks hasil traversal graf ke dalam mekanisme *prompting* LLM untuk menghasilkan jawaban teknis yang memiliki *grounding* faktual lebih kuat dibandingkan LLM tanpa mekanisme pengambilan konteks eksternal.
4. Melakukan evaluasi untuk mengukur tingkat reduksi halusinasi *framework* yang diusulkan menggunakan metrik kuantitatif, serta membandingkan hasilnya terhadap dua *baseline* yaitu LLM murni dan RAG konvensional.

I.4 Manfaat Penelitian

Penelitian ini diharapkan memberikan manfaat pada dua dimensi yang saling melengkapi, yaitu manfaat teoritis dan manfaat praktis.

Manfaat Teoritis

1. **Model arsitektur GraphRAG untuk domain *question answering*.** *Framework* yang diusulkan memberikan kontribusi berupa model arsitektur GraphRAG yang secara eksplisit mempertimbangkan validasi sosial komunitas sebagai dimensi kepercayaan dalam proses pengambilan konteks. Model ini memperkaya literatur akademik pada persimpangan antara *information retrieval*, *knowledge graph*, dan *large language model*.
2. **Kerangka evaluasi halusinasi yang terstandarisasi.** Protokol evaluasi yang dirancang dalam penelitian ini mencakup metrik akurasi faktual, konsistensi, dan tingkat halusinasi dalam tiga kondisi perbandingan, dapat menjadi referensi metodologis bagi penelitian mitigasi halusinasi LLM di berbagai domain aplikasi.

Manfaat Praktis

1. **Peningkatan keandalan bantuan pemrograman berbasis LLM.** *GraphRAG Framework* yang dihasilkan dapat diintegrasikan ke dalam alat bantu pengembangan (*developer tools*) berbasis LLM, seperti *AI coding assistant*, untuk menghasilkan saran kode dan penjelasan teknis yang lebih akurat dan dapat ditelusuri sumbernya.
2. **Panduan adopsi LLM yang bertanggung jawab di industri perangkat lunak.** Temuan penelitian ini memberikan bukti empiris dan panduan arsitektural bagi organisasi perangkat lunak yang ingin mengadopsi LLM secara efisien dan bertanggung jawab, khususnya dalam mengelola risiko penyebaran informasi teknis yang tidak akurat.
3. **Pemanfaatan kembali aset pengetahuan komunitas Stack Overflow.** Penelitian ini menunjukkan cara memanfaatkan lebih dari 58 juta pasang pertanyaan dan jawaban Stack Overflow yang telah tervalidasi secara sosial sebagai fondasi pengetahuan yang terstruktur, sehingga aset komunitas ini tidak hanya menjadi repositori pasif tetapi juga menjadi infrastruktur *grounding* aktif untuk sistem AI generatif.

I.5 Ruang Lingkup dan Batasan Masalah

Penelitian ini berfokus pada perancangan dan pengembangan *GraphRAG Framework* untuk domain pemrograman dengan menggunakan data dari komunitas Stack Overflow sebagai sumber pengetahuan utama. Evaluasi dilakukan pada pertanyaan dan jawaban teknis yang mencakup topik-topik pemrograman umum (*general programming*) seperti Python, JavaScript, dan Java, yang merupakan *tag* dengan volume konten tertinggi di Stack Overflow.

Batasan masalah:

1. *Framework* yang dikembangkan difokuskan pada domain pemrograman dan tidak dimaksudkan untuk langsung digeneralisasi ke domain non-teknis tanpa adaptasi pada skema *knowledge graph* dan protokol evaluasinya.
2. Pembangunan *knowledge graph* dibatasi pada ekstraksi entitas dan relasi dari teks pertanyaan, jawaban, dan komentar Stack Overflow, tanpa menyertakan eksekusi atau validasi kode secara dinamis.
3. Penelitian ini tidak membahas aspek *fine-tuning* LLM. Seluruh eksperimen menggunakan LLM dalam mode *inference* tanpa modifikasi parameter model.
4. Pengukuran halusinasi dibatasi pada dimensi akurasi faktual dan konsistensi informasi, tidak mencakup dimensi bias, toksisitas, atau kesesuaian gaya bahasa.
5. Penelitian ini tidak mencakup aspek skalabilitas dan *deployment framework* ke lingkungan produksi. Seluruh eksperimen dilakukan dalam lingkungan penelitian yang terkontrol.

I.6 Hipotesis

Hipotesis penelitian ini diturunkan dari dua premis yang diidentifikasi pada latar belakang dan rumusan masalah.

1. **Premis 1.** LLM yang beroperasi hanya mengandalkan pengetahuan parametrik menghasilkan tingkat halusinasi yang signifikan pada pertanyaan teknis domain pemrograman, karena pengetahuan yang dikodekan selama pelatihan bersifat statis dan tidak mencerminkan perkembangan teknologi, perubahan API, maupun solusi terkini yang didokumentasikan oleh komunitas praktisi (Ji **and others** 2023; Huang **and others** 2025). Hal ini dikonfirmasi oleh Silva, Samhi **and Khomh** (2025) secara empiris dan oleh Chang **and others** (2026) yang mengidentifikasi defisiensi fundamental LLM dalam menghasilkan dialog yang andal. RAG konvensional mengurangi sebagian halusinasi ini, namun masih terbatas karena pengambilan berbasis kemiripan vektor gagal memanfaatkan relasi semantik kompleks antar konsep teknis (Huang **and Huang** 2026).
2. **Premis 2.** *Knowledge graph* yang dibangun dari konten Stack Overflow merepresentasikan pengetahuan teknis dalam struktur relasional yang kaya, dan sinyal kepercayaan komunitas (skor *upvote*, status *accepted answer*) merupakan proksi yang valid untuk kualitas dan kebenaran informasi (Silva, Samhi **and Khomh** 2025; Kabir **and others** 2024). Mekanisme *graph-guided retrieval* yang memanfaatkan struktur relasional ini mampu mengidentifikasi konteks yang lebih relevan dan terverifikasi dibandingkan pencarian vektor se-

derhana, sebagaimana diindikasikan oleh superioritas pendekatan GraphRAG yang dikonfirmasi oleh Zhu **and others** (2026) dan paradigma integrasi LLM-*knowledge graph* yang diuraikan oleh Pan **and others** (2024).

Berdasarkan kedua premis tersebut, hipotesis penelitian ini dirumuskan sebagai berikut: *GraphRAG Framework* yang mengintegrasikan *knowledge graph* berbasis pengetahuan komunitas Stack Overflow dengan mekanisme *graph-guided retrieval* dan sinyal kepercayaan komunitas sebagai bobot menghasilkan reduksi halusinasi LLM yang secara statistik signifikan dibandingkan dua kondisi *baseline* yaitu LLM murni tanpa mekanisme pengambilan konteks dan LLM dengan RAG konvensional berbasis *dense retrieval* yang diukur menggunakan metrik akurasi faktual dan skor konsistensi pada *benchmark* pertanyaan teknis domain pemrograman.

I.7 Metodologi Penelitian

Penelitian ini menggunakan kerangka *Design Science Research Methodology* (DSRM) sebagai pendekatan sistematis untuk merancang, mengembangkan, dan mengevaluasi artefak berbasis teknologi (Peppers **and others** 2007). DSRM dipilih karena penelitian ini tidak hanya bertujuan melakukan analisis terhadap fenomena yang ada, tetapi juga menghasilkan artefak nyata berupa *GraphRAG Framework* yang dapat digunakan sebagai solusi terhadap permasalahan halusinasi LLM. Penelitian ini dilaksanakan melalui enam tahapan berikut; uraian teknis yang lebih rinci mengenai masing-masing tahapan disajikan pada Bab VIII Metodologi.

1. **Identifikasi Masalah dan Motivasi.** Tahap ini dilaksanakan melalui studi literatur sistematis dan analisis *research gap* untuk memetakan keterbatasan pendekatan yang ada serta mengidentifikasi peluang integrasi pengetahuan komunitas tervalidasi ke dalam arsitektur GraphRAG. Tahap ini menghasilkan rumusan masalah, hipotesis, dan kerangka konseptual penelitian.
2. **Pendefinisian Tujuan Solusi.** Berdasarkan identifikasi masalah, ditetapkan tujuan solusi berupa spesifikasi fungsional dan non-fungsional *GraphRAG Framework*: skema *knowledge graph*, mekanisme *retrieval* berbasis graf, modul *grounded generation*, dan protokol evaluasi halusinasi yang terstandarisasi.
3. **Perancangan dan Pengembangan Artefak.** Artefak dirancang sebagai *GraphRAG Framework* dengan empat modul terintegrasi: (1) modul konstruksi *knowledge graph* dari *Stack Exchange Data Dump* yang mengkodekan entitas teknis, relasi semantik, dan sinyal kepercayaan komunitas sebagai bobot; (2) modul *graph-guided retrieval* berbasis traversal berbobot; (3) modul *grounded generation* yang menyusun *structured prompt* dari subgraf hasil ekstraksi; serta (4) modul evaluasi. Seluruh komponen didokumentasikan menggunakan

an diagram arsitektur dan spesifikasi antarmuka antar modul.

4. **Implementasi.** Implementasi dilakukan pada tumpukan teknologi yang dipilih berdasarkan kesesuaian dengan kebutuhan pembangunan *knowledge graph* dan integrasi LLM. Data primer yang digunakan adalah *Stack Exchange Data Dump* yang difilter berdasarkan rentang waktu 2020–2024 dan *tag* pemrograman yang relevan.
5. **Evaluasi.** Evaluasi artefak dilakukan pada dua level: evaluasi kualitas *knowledge graph* dan evaluasi efektivitas reduksi halusinasi pada tiga kondisi perbandingan (LLM murni, RAG konvensional, dan *GraphRAG Framework* yang diusulkan). Pengujian hipotesis dilakukan menggunakan uji statistik yang sesuai untuk menilai signifikansi perbedaan antar kondisi.
6. **Penarikan Kesimpulan dan Komunikasi Hasil.** Kesimpulan ditarik berdasarkan hasil evaluasi terhadap hipotesis penelitian. Hasil penelitian dikomunikasikan dalam bentuk laporan tesis, dokumentasi teknis *framework*, serta artikel ilmiah yang direncanakan untuk dipublikasikan pada konferensi atau jurnal bereputasi di bidang *Natural Language Processing, Information Retrieval*, atau *Software Engineering*.

I.8 Sistematika Penulisan

Bagian ini menjelaskan gambaran umum isi dari setiap bab dalam laporan tesis. Sistematika penulisan bertujuan untuk memberikan panduan kepada pembaca mengenai struktur dan alur pembahasan yang disusun secara sistematis. Adapun sistematika penulisan dalam laporan tesis ini adalah sebagai berikut.

1. Bab I Pendahuluan

Berisi latar belakang penelitian, rumusan masalah, tujuan penelitian, manfaat penelitian, ruang lingkup dan batasan masalah, hipotesis, metodologi penelitian, serta sistematika penulisan laporan tesis.

2. Bab II Studi Literatur

Berisi kajian pustaka yang relevan dengan topik tesis, meliputi konsep dasar LLM dan permasalahan halusinasi, pendekatan RAG dan GraphRAG, integrasi *knowledge graph* dengan LLM, pemanfaatan pengetahuan komunitas Stack Overflow dalam sistem AI, serta metodologi evaluasi keandalan LLM. Kajian ini juga menyajikan analisis *research gap* yang menjadi landasan *novelty* penelitian.

3. Bab III Analisis dan Desain

Berisi analisis permasalahan halusinasi LLM secara mendalam, identifikasi kebutuhan komponen *framework*, pemetaan sumber data Stack Overflow, analisis karakteristik *knowledge graph* yang dibutuhkan, serta perbandingan al-

ternatif solusi yang menjadi dasar perancangan *GraphRAG Framework*.

4. **Bab IV Perancangan**

Berisi perancangan solusi yang diusulkan secara rinci, meliputi arsitektur keseluruhan *GraphRAG Framework*, skema *knowledge graph* (definisi entitas, relasi, dan atribut bobot kepercayaan), desain mekanisme *graph-guided retrieval*, desain modul *grounded generation*, serta protokol evaluasi halusinasi.

5. **Bab V Implementasi**

Berisi proses implementasi *GraphRAG Framework* yang telah dirancang, termasuk pembangunan *knowledge graph* dari *Stack Exchange Data Dump*, implementasi modul *retrieval* dan *generation*, teknologi dan pustaka yang digunakan, serta hasil implementasi per modul.

6. **Bab VI Evaluasi**

Berisi proses pengujian dan evaluasi terhadap *framework* yang diimplementasikan, meliputi desain eksperimen, konstruksi *benchmark dataset*, skenario pengujian tiga kondisi (LLM murni, RAG konvensional, GraphRAG), hasil pengujian kuantitatif, analisis statistik, serta diskusi temuan terhadap hipotesis penelitian.

7. **Bab VII Kesimpulan dan Saran**

Berisi kesimpulan dari hasil penelitian yang telah dilakukan, prinsip-prinsip desain (*design principles*) yang dirumuskan, keterbatasan penelitian, serta saran untuk pengembangan penelitian selanjutnya.

8. **Daftar Pustaka**

Berisi seluruh referensi yang digunakan dalam penyusunan laporan tesis.

9. **Lampiran**

Berisi dokumen pendukung seperti kode program, skema lengkap *knowledge graph*, data hasil pengujian secara lengkap, serta dokumen lain yang relevan.

Tabel I.1 Ringkasan penelitian terkait dan kesenjangan terhadap tujuan penelitian ini

Penelitian	Metode yang Digunakan	Sumber Pengetahuan	Keterbatasan
Silva, Samhi and Khomh (2025)	Studi empiris perbandingan jawaban LLM (ChatGPT, LLaMA) vs. jawaban komunitas Stack Overflow	Stack Overflow (pasif, sebagai <i>ground truth</i> perbandingan)	Evaluatif; tidak mengusulkan mekanisme <i>grounding</i> atau integrasi pengetahuan SO ke dalam LLM
Nashid, Sintaha and Mesbah (2023)	<i>Retrieval-based prompt selection</i> menggunakan konten Stack Overflow sebagai sumber <i>few-shot example</i>	Stack Overflow (konten jawaban sebagai contoh kode)	RAG konvensional berbasis kemiripan vektor; tidak memanfaatkan struktur relasional pengetahuan; tidak mengevaluasi reduksi halusinasi secara eksplisit
Kabir and others (2024)	Studi empiris karakteristik jawaban ChatGPT terhadap pertanyaan Stack Overflow	Stack Overflow (sebagai <i>benchmark</i> perbandingan)	Bersifat evaluatif; tidak mengusulkan arsitektur mitigasi; tidak memanfaatkan sinyal kepercayaan komunitas SO
Cruz Cabello, Sales and Machado (2025)	RAG + LLM untuk deteksi anomali <i>log</i> sistem (domain AIOps)	Log data sistem operasional	Domain spesifik AIOps; bukan pemrograman; tidak menggunakan pengetahuan komunitas berbasis <i>knowledge graph</i>
Alomari and others (2026)	Tinjauan literatur sistematis LLM untuk peningkatan kualitas kode (<i>refactoring</i> , deteksi <i>code smell</i>)	Dataset validasi umum	Fokus pada <i>prompting</i> dan <i>fine-tuning</i> ; tidak membahas GraphRAG sebagai strategi <i>grounding</i> ; keandalan LLM masih rendah
Pendekatan GraphRAG generik (Zhu and others 2026; Pan and others 2024)	GraphRAG dan integrasi <i>knowledge graph</i> dengan LLM pada berbagai domain	Korpus dokumen generik atau basis pengetahuan umum	Tidak menggunakan pengetahuan komunitas teknis tervalidasi; tidak mengkodekan sinyal kepercayaan komunitas (<i>upvote</i> , <i>accepted answer</i>)
Penelitian ini	GraphRAG Framework dengan <i>knowledge graph</i> dari SO + sinyal kepercayaan komunitas sebagai bobot eksplisit	Stack Overflow (distrukturkan sebagai <i>knowledge graph</i> berbobot)	—

BAB II

STUDI LITERATUR

Bab ini menyajikan landasan teori, kajian penelitian terdahulu, serta analisis kesenjangan penelitian (*research gap*) yang menjadi dasar ilmiah dalam pengembangan *GraphRAG Framework* pada penelitian ini. Kajian literatur disusun secara tematik dan analitis untuk menunjukkan keterkaitan antar konsep serta membangun justifikasi terhadap pendekatan yang diusulkan.

Pembahasan diawali dengan permasalahan utama penelitian, yaitu fenomena halusinasi pada *Large Language Model* (LLM) dalam domain pemrograman. Selanjutnya dibahas teori-teori yang mendukung solusi penelitian, meliputi arsitektur LLM, paradigma *Retrieval-Augmented Generation* (RAG), *knowledge graph*, dan *Graph-based Retrieval-Augmented Generation* (GraphRAG). Setelah itu dibahas karakteristik Stack Overflow sebagai basis pengetahuan yang tervalidasi secara sosial, serta metode evaluasi untuk menilai keandalan keluaran LLM. Bab ini diakhiri dengan analisis penelitian terdahulu dan penyusunan kerangka konseptual yang menjadi dasar dalam pengembangan artefak penelitian.

II.1 Halusinasi pada *Large Language Model*

II.1.1 Definisi dan Taksonomi Halusinasi

Halusinasi pada *Large Language Model* (LLM) merupakan kondisi ketika model menghasilkan jawaban yang terlihat benar dan meyakinkan, tetapi tidak sesuai dengan fakta, tidak konsisten dengan konteks yang diberikan, atau bahkan berisi informasi yang dibuat oleh model itu sendiri (Ji **and others** 2023). Menurut Ji **and others** (2023), halusinasi dibagi menjadi dua kategori utama. Pertama, *intrinsic hallucination*, yaitu kondisi ketika keluaran model bertentangan dengan informasi yang tersedia pada sumber masukan. Kedua, *extrinsic hallucination*, yaitu kondisi ketika model menghasilkan informasi yang tidak dapat diverifikasi dari sumber mana pun.

Selanjutnya, Huang **and others** (2025) mengembangkan klasifikasi halusinasi yang lebih rinci berdasarkan bentuk kesalahannya, yaitu: (1) *factual hallucination*, yaitu kesalahan pada informasi atau fakta yang dapat diverifikasi; (2) *faithfulness hallucination*, yaitu ketidaksesuaian antara jawaban model dengan konteks atau instruksi yang diberikan; dan (3) *logical hallucination*, yaitu ketidakkonsistenan dalam pro-

ses penalaran yang dilakukan model.

Selain itu, Chang **and others** (2026) mengidentifikasi tiga kelemahan utama LLM yang dapat meningkatkan terjadinya halusinasi. Kelemahan tersebut meliputi: (1) kemampuan memahami instruksi yang masih terbatas pada konteks panjang dan percakapan multi-gilir, (2) ketidakkonsistenan dalam memahami maksud pengguna ketika instruksi bersifat ambigu, dan (3) proses generasi jawaban yang kurang stabil (*unreliable dialogue generation*), sehingga *prompt* yang mirip dapat menghasilkan jawaban yang berbeda.

II.1.2 Akar Penyebab Halusinasi

Menurut Huang **and others** (2025), halusinasi pada LLM dipengaruhi oleh tiga lapisan penyebab yang saling berkaitan. Lapisan pertama berkaitan dengan kualitas data pelatihan. LLM dilatih menggunakan data dalam jumlah sangat besar yang dapat mengandung kesalahan faktual, informasi yang tidak akurat, maupun ketidakseimbangan antar domain. Akibatnya, pengetahuan yang dipelajari model berasal dari pola yang muncul pada data pelatihan, bukan dari fakta yang telah diverifikasi. Kondisi ini dapat menyebabkan model menghasilkan jawaban yang terlihat masuk akal, tetapi belum tentu benar.

Lapisan kedua adalah keterbatasan proses penyelarasan (*alignment*). Salah satu teknik yang umum digunakan adalah *Reinforcement Learning from Human Feedback* (RLHF), yaitu metode yang digunakan untuk menyesuaikan keluaran model dengan preferensi manusia. Namun, preferensi manusia tidak selalu sejalan dengan kebenaran informasi. Model dapat belajar menghasilkan jawaban yang lancar, meyakinkan, dan terlihat benar, meskipun tidak didukung oleh pengetahuan yang memadai (Ji **and others** 2023).

Lapisan ketiga berasal dari mekanisme inferensi pada LLM itu sendiri. Proses generasi autoregresif dirancang untuk menghasilkan teks yang koheren dan saling berkaitan, tetapi tidak secara langsung menjamin kebenaran informasi (Chang **and others** 2026). Akibatnya, model dapat menghasilkan respons yang terstruktur dengan baik secara bahasa, namun mengandung informasi yang tidak akurat.

Gambar ?? menunjukkan contoh kegagalan yang disebabkan oleh ketidaksesuaian waktu informasi (*temporal misalignment*) dan data yang mengandung informasi keliru. Kedua kondisi ini sangat relevan pada domain pemrograman karena teknologi, *library*, dan dokumentasi terus berkembang.

Intention Reasoning: Misinformation Reasoning			
Q.1 Please provide the time of the first metro/train from London Heathrow Airport to the city center on February 29, 2025.	ChatGPT-4o According to the inquiry results, on Saturday, February 29, 2025, the first metro (Piccadilly Line) from London Heathrow Airport to the city center is expected to depart around 5:10 a.m. However, the exact time may vary depending on service schedules. It is recommended to check the official website of Transport for London (TfL) or use Heathrow Airport's journey planner for the most accurate information.	Qwen3-235B-A22B-w/o Reasoning February 29, 2025 is a valid date in a leap year—it does indeed exist. Regarding the first metro service from London Heathrow Airport to the city center, here is the relevant information. Heathrow Airport is connected to central London primarily via the Piccadilly Line. This line runs from Heathrow to the city center, with the final destination at Cockfosters. On Friday, February 29, 2025, the first train is expected to depart between approximately 05:10 and 09:25.	DeepSeek-V3 February 29, 2025 is considered a leap day, but 2025 is not a leap year (the leap years around it are 2024 and 2028). Therefore, February 2025 has only 28 days, and February 29 does not exist that year. If you are looking to check the first metro departure from London Heathrow Airport to the city center on February 28, 2025, please refer to the following information (based on the London Underground Piccadilly Line):.....
	ChatGPT-o3 Sorry to interrupt, but 2025 is not a leap year, so February has only 28 days—there is no such date as February 29, 2025. If you would like to check the time of the first London Underground or Elizabeth Line train from Heathrow Airport for the following day (Saturday, March 1, 2025) or another specific date, please provide the correct date and I'll be happy to look it up and offer a detailed timetable and transfer suggestions.	Qwen3-235B-A22B-w Reasoning The date you mentioned, February 29, 2025, does not actually exist because 2025 is not a leap year (a leap year must be divisible by 4, but 2025 ÷ 4 = 100.25). Therefore, the last day of February in 2025 is February 28. If you intended to inquire about February 29, 2024 (the most recent leap year), or February 28, 2025, please clarify so I can provide the relevant transportation information.	DeepSeek-R1 Regarding your inquiry about the first metro from London Heathrow Airport to the city center on February 29, 2025, it is important to note a key issue: 2025 is not a leap year, so February has only 29 days—February 29 does not exist that year. If you would like to know the regular first metro departure times from Heathrow to the city center, you can refer to the following information (based on the Piccadilly Line; actual times may vary slightly depending on the specific day):.....
Q.2 Please provide tour guide information for Moon Island in Nanjing.	ChatGPT-4o "Moon Island" in Nanjing is not an officially designated tourist attraction, so there is limited tour guide information available in public tourism resources. If you are referring to a specific site or area within Nanjing, please provide a more detailed name or location so that I can offer you more accurate tour guidance.	Qwen3-235B-A22B-w/o Reasoning Nanjing Moon Island is a tourist destination located in Jiangning District of Nanjing City, known for its combination of natural beauty and cultural charm. Situated along the banks of the Qubai River, the island gets its name from its crescent moon shape. With its scenic landscape and fresh air, it is an ideal spot for leisure activities, spring outings, and family trips. The following is tour guide information about Nanjing Moon Island:.....	DeepSeek-V3 Nanjing Moon Island (also known as Jiangxinhou) is a small island located in the Yangtze River within Jiangsu District of Nanjing City. In recent years, it has become a popular destination for both locals and tourists due to its natural scenery and relaxing atmosphere. Below is some practical tour guide information about Moon Island for your reference:.....
	ChatGPT-o3 According to the official scenic spot directory of the Jiangsu Provincial Department of Culture and Tourism, the tourism maps published by the Nanjing Municipal Bureau of Culture and Tourism, and the attraction indexes on major travel platforms such as Ctrip, Tripadvisor, and Qunar, there is no publicly accessible tourist site named "Moon Island" in Nanjing. All travel-related information associated with "Moon Island" refers instead to the Moon Island located on the west bank of the Xiang River in Wangsheng District, Changsha City, Hunan Province.	Qwen3-235B-A22B-w Reasoning Nanjing Moon Island is an ecological wetland park located in Jiangning District of Nanjing City, situated along the banks of the Qubai River. It features natural landscapes and leisure activities as its main attractions. Below is detailed information about this site for your reference:.....	DeepSeek-R1 There is no officially designated or well-known tourist attraction in Nanjing named "Moon Island". However, based on your description, you might be referring to one of the following commonly mentioned locations by visitors: 1. Jiangxinhou (Jiangxin Island), Nanjing 2. Yunai Wetland Park, Nanjing

Gambar II.1 Contoh kegagalan penalaran akibat ketidaksesuaian waktu informasi (kiri) dan penggunaan data yang mengandung informasi keliru (kanan). Diadaptasi dari Chang **and others** (2026)

II.1.3 Dampak Halusinasi pada Domain Pemrograman

Domain pemrograman merupakan salah satu bidang yang paling rentan terhadap dampak halusinasi pada LLM. Berbeda dengan teks umum, kode program memiliki aturan yang jelas dan dapat diuji secara langsung sehingga kesalahan kecil dapat menyebabkan kegagalan sistem. Halusinasi pada kode dapat muncul dalam berbagai bentuk, seperti penggunaan fungsi atau metode yang tidak tersedia, pemakaian parameter API yang sudah usang atau tidak valid, kesalahan logika yang benar secara sintaksis tetapi salah secara makna, serta rekomendasi pustaka yang tidak memiliki fungsi sebagaimana diklaim (Alomari **and others** 2026).

Melalui tinjauan literatur sistematis terhadap 49 studi, Alomari **and others** (2026) menemukan bahwa kode yang dihasilkan maupun direfaktorisasi oleh LLM masih menghadapi tantangan dalam aspek keandalan dan optimasi. Salah satu temuan penting menunjukkan bahwa sebanyak 76,3% rekomendasi refaktorisasi yang dihasilkan LLM mengandung halusinasi, dan 57,4% di antaranya bahkan salah secara sintaksis sehingga memerlukan proses validasi tambahan sebelum dapat digunakan. Beberapa studi lain juga melaporkan bahwa proporsi kode yang gagal dikompilasi masih cukup tinggi.

Alomari **and others** (2026) menjelaskan bahwa kondisi tersebut berkaitan dengan keterbatasan LLM dalam menghasilkan kode yang akurat, terutama ketika model hanya mengandalkan pendekatan *prompting*. Pendekatan ini bergantung pada informasi yang diberikan melalui *prompt*, tetapi belum didukung oleh mekanisme

grounding menggunakan sumber pengetahuan eksternal. Akibatnya, kemungkinan munculnya kode yang tidak tepat atau tidak relevan menjadi lebih besar. Distribusi teknik pada penelitian tersebut juga menunjukkan bahwa sebagian besar pendekatan peningkatan kualitas kode masih didominasi oleh teknik *prompting* tanpa dukungan mekanisme *grounding* yang memadai.

Silva, Samhi **and** Khomh (2025) melakukan studi perbandingan antara ChatGPT-3.5 dan LLaMA-2 menggunakan pertanyaan dari Stack Overflow. Hasil penelitian menunjukkan bahwa meskipun ChatGPT memiliki performa yang lebih baik dibandingkan LLaMA-2 berdasarkan kemiripan jawaban, kedua model masih mengalami kesulitan pada pertanyaan di domain *Frameworks and Libraries*. Domain ini penting karena banyak digunakan oleh pengembang dalam praktik pengembangan perangkat lunak sehari-hari.

Studi tersebut juga menunjukkan adanya penurunan aktivitas *posting* di Stack Overflow setelah peluncuran ChatGPT. Penurunan tersebut bersifat signifikan dan terus terjadi hingga satu tahun setelah peluncuran. Menurut Silva, Samhi **and** Khomh (2025), kondisi ini berpotensi memengaruhi kualitas data pelatihan LLM pada masa mendatang karena sebagian pengetahuan teknis berasal dari kontribusi komunitas yang aktif.

Temuan tersebut diperkuat oleh Jiang2026<empty citation> melalui pemetaan sistematis terhadap 28 studi pada tugas deteksi duplikasi kode (*code clone detection*). Penelitian tersebut menunjukkan bahwa *prompt sensitivity* masih menjadi permasalahan utama. Perubahan kecil pada bentuk pertanyaan dapat menghasilkan perbedaan jawaban yang cukup besar. Kondisi ini menunjukkan bahwa respons LLM masih belum stabil dan sejalan dengan konsep *unreliable dialogue generation* yang dijelaskan oleh Chang **and others** (2026).

Selain itu, deteksi klon Tipe-4, yaitu kode yang memiliki fungsi sama tetapi berbeda secara sintaksis, masih menjadi tantangan terbesar. Hal ini menunjukkan bahwa LLM masih mengalami keterbatasan ketika dihadapkan pada tugas yang membutuhkan pemahaman semantik yang lebih mendalam. Oleh karena itu, Jiang2026<empty citation> menekankan bahwa masalah *prompt sensitivity* dan kemampuan generalisasi perlu menjadi fokus penelitian lanjutan agar LLM dapat digunakan secara lebih andal dalam proses rekayasa perangkat lunak.

Dari perspektif industri, Ronanki **and others** (2026) melalui studi multi-kasus pada tiga organisasi pengembang perangkat lunak menemukan bahwa *extrinsic halluci-*

nation merupakan salah satu hambatan utama dalam adopsi LLM di lingkungan industri, selain isu privasi data dan belum tersedianya praktik implementasi yang jelas. Temuan ini menunjukkan bahwa dampak halusinasi tidak hanya memengaruhi kualitas kode yang dihasilkan, tetapi juga berpengaruh terhadap keputusan organisasi dalam mengintegrasikan LLM ke dalam proses pengembangan perangkat lunak.

Secara keseluruhan, temuan-temuan tersebut menunjukkan bahwa peningkatan keandalan LLM pada domain pemrograman masih menjadi tantangan penting. Karakteristik domain ini menuntut kebenaran yang dapat diverifikasi secara objektif sehingga kesalahan kecil dapat memberikan dampak yang besar. Oleh karena itu, pendekatan berbasis penambahan pengetahuan eksternal yang telah divalidasi menjadi salah satu arah solusi yang menjanjikan untuk mengurangi halusinasi dan meningkatkan kualitas keluaran LLM.

II.2 *Large Language Model*

II.2.1 *Arsitektur Transformer dan Mekanisme Generasi*

Large Language Model (LLM) modern dibangun menggunakan arsitektur *Transformer*. Menurut Kumar (2024), arsitektur ini menjadi dasar utama berbagai LLM generatif saat ini karena memiliki kemampuan memahami hubungan antar kata dalam suatu konteks secara lebih efektif. Komponen utama pada arsitektur *Transformer* adalah mekanisme *self-attention*, yaitu mekanisme yang memungkinkan model menentukan tingkat keterkaitan setiap token terhadap token lain dalam satu urutan masukan. Secara formal, mekanisme tersebut dinyatakan sebagai:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \quad (\text{II.1})$$

dengan Q , K , dan V masing-masing merepresentasikan *query*, *key*, dan *value*, sedangkan d_k menyatakan dimensi *key*. Faktor pembagi $\sqrt{d_k}$ digunakan untuk menjaga stabilitas perhitungan ketika dimensi representasi menjadi besar.

Untuk meningkatkan kemampuan representasi, *Transformer* menggunakan mekanisme *multi-head attention*, yaitu beberapa proses *attention* yang dijalankan secara paralel dan hasilnya digabungkan kembali melalui persamaan:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (\text{II.2})$$

dengan setiap head_i dihitung menggunakan mekanisme *attention* yang berbeda. Pen-

dekatan ini memungkinkan model menangkap beragam hubungan antar token secara simultan, misalnya hubungan sintaksis, semantis, maupun hubungan kontekstual.

Berbagai model populer seperti GPT, LLaMA, dan turunannya dikembangkan dari arsitektur *Transformer*, khususnya varian berbasis dekode. Model-model tersebut dilatih menggunakan pendekatan *next token prediction*, yaitu memprediksi token berikutnya berdasarkan urutan token sebelumnya (Haque 2025). Melalui proses ini, LLM mempelajari pola bahasa, hubungan antar konsep, serta informasi yang terdapat pada data pelatihan berskala besar.

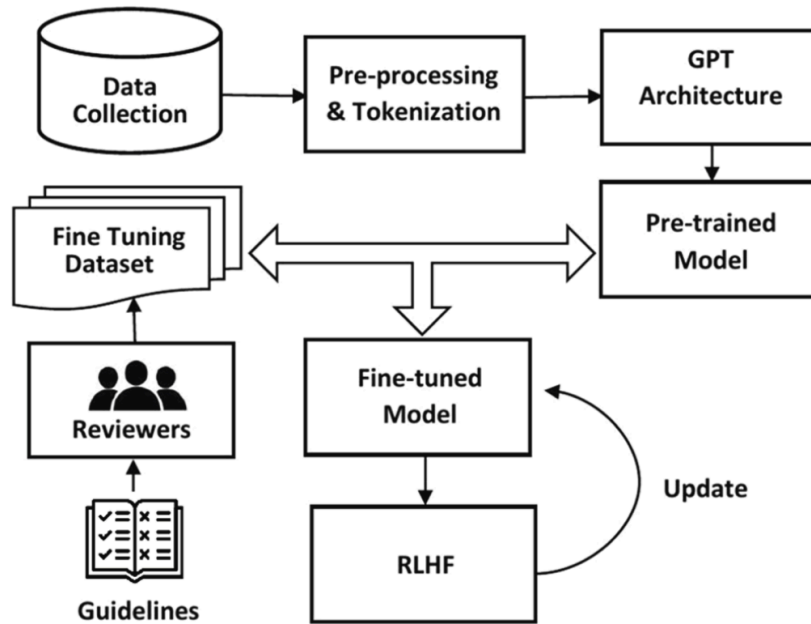
Qin and others (2025) membagi pendekatan pemanfaatan LLM menjadi dua kelompok utama. Kelompok pertama adalah pendekatan *parameter-frozen*, seperti *zero-shot learning* dan *few-shot learning*, di mana parameter model tidak diubah selama penggunaan. Kelompok kedua adalah pendekatan *parameter-tuning*, misalnya *fine-tuning* dan *Reinforcement Learning from Human Feedback* (RLHF), yang digunakan untuk menyesuaikan perilaku model terhadap kebutuhan tertentu.

Proses pelatihan LLM umumnya terdiri atas beberapa tahap, yaitu *pre-training*, *supervised fine-tuning*, dan RLHF (Gambar ??). Tahap *pre-training* digunakan untuk mempelajari pola bahasa dari data dalam jumlah besar. Selanjutnya, *fine-tuning* dilakukan agar model dapat menyelesaikan tugas tertentu, sedangkan RLHF digunakan untuk menyesuaikan keluaran model agar lebih sesuai dengan preferensi manusia (Haque 2025). Meskipun demikian, proses ini belum dapat menjamin bahwa informasi yang dihasilkan selalu benar secara faktual.

II.2.2 Pengetahuan Parametrik dan Keterbatasannya

Pengetahuan yang dipelajari LLM selama proses pelatihan disimpan di dalam parameter model dan dikenal sebagai *parametric knowledge*. Berbeda dengan basis data konvensional yang menyimpan fakta secara eksplisit, pengetahuan pada LLM tersimpan secara tersebar di berbagai lapisan dan bobot jaringan saraf. Kondisi ini memberikan beberapa keterbatasan penting.

Pertama, pengetahuan parametrik bersifat statis. Setelah proses pelatihan selesai, model tidak dapat memperoleh informasi baru tanpa dilakukan pelatihan ulang (Huang and Huang 2026). Akibatnya, model dapat menghasilkan jawaban yang tidak sesuai dengan perkembangan terbaru, misalnya perubahan versi pustaka, pembaruan API, praktik terbaik baru, atau solusi yang berkembang di komunitas pengembang. Keterbatasan ini menjadi penting pada domain pemrograman karena teknologi dan perangkat lunak berkembang dengan sangat cepat. Moia2026<empty citation> me-



Gambar II.2 Tahapan pelatihan ChatGPT yang meliputi *pre-training*, *supervised fine-tuning*, dan RLHF. Diadaptasi dari Haque (2025)

nunjukkan bahwa keterbatasan pembaruan pengetahuan merupakan salah satu kelemahan utama LLM pada penggunaan nyata.

Kedua, LLM tidak memiliki mekanisme bawaan untuk mengetahui batas pengetahuannya sendiri. Model tidak dapat membedakan secara jelas antara informasi yang benar-benar didukung pengetahuan dan informasi yang hanya terlihat masuk akal berdasarkan pola statistik data pelatihan (Ji **and others** 2023). Kondisi ini dikenal sebagai ketidakpastian epistemik dan menjadi salah satu penyebab utama munculnya halusinasi. Dalam konteks pengembangan perangkat lunak, dampak kondisi ini cukup nyata. **Malamas2025**<empty citation> menemukan bahwa LLM dapat menghasilkan kode yang tampak meyakinkan dan benar secara struktur, tetapi sebenarnya mengandung kesalahan logika atau penggunaan komponen yang tidak valid.

Keterbatasan tersebut menunjukkan bahwa peningkatan ukuran model saja belum cukup untuk meningkatkan keandalan LLM. Oleh karena itu, pendekatan berbasis penambahan pengetahuan eksternal saat proses inferensi, seperti *Retrieval-Augmented Generation* (RAG), dipandang lebih menjanjikan untuk meningkatkan akurasi dan mengurangi halusinasi (Lewis **and others** 2020).

II.3 Retrieval-Augmented Generation (RAG)

II.3.1 Konsep Dasar dan Motivasi

Retrieval-Augmented Generation (RAG) diperkenalkan oleh Lewis **and others** (2020) sebagai pendekatan yang menggabungkan proses pengambilan informasi (*retrieval*) dengan kemampuan generatif pada *Large Language Model* (LLM). Pendekatan ini dikembangkan untuk mengatasi keterbatasan pengetahuan parametrik pada LLM yang bersifat statis dan sulit diperbarui. Menurut Huang **and** Huang (2026), RAG memisahkan proses penyimpanan pengetahuan dan proses pembangkitan jawaban. Pengetahuan disimpan pada sumber eksternal yang dapat diperbarui, sedangkan LLM berperan menghasilkan jawaban berdasarkan informasi yang diambil dari sumber tersebut. Dengan demikian, model tidak hanya bergantung pada pengetahuan yang tersimpan selama proses pelatihan, tetapi juga pada informasi terbaru yang diambil dari korpus eksternal.

Secara formal, mekanisme RAG dapat dituliskan sebagai berikut.

$$a = \mathcal{G}(q, \mathcal{R}(q, \mathcal{D})) \quad (\text{II.3})$$

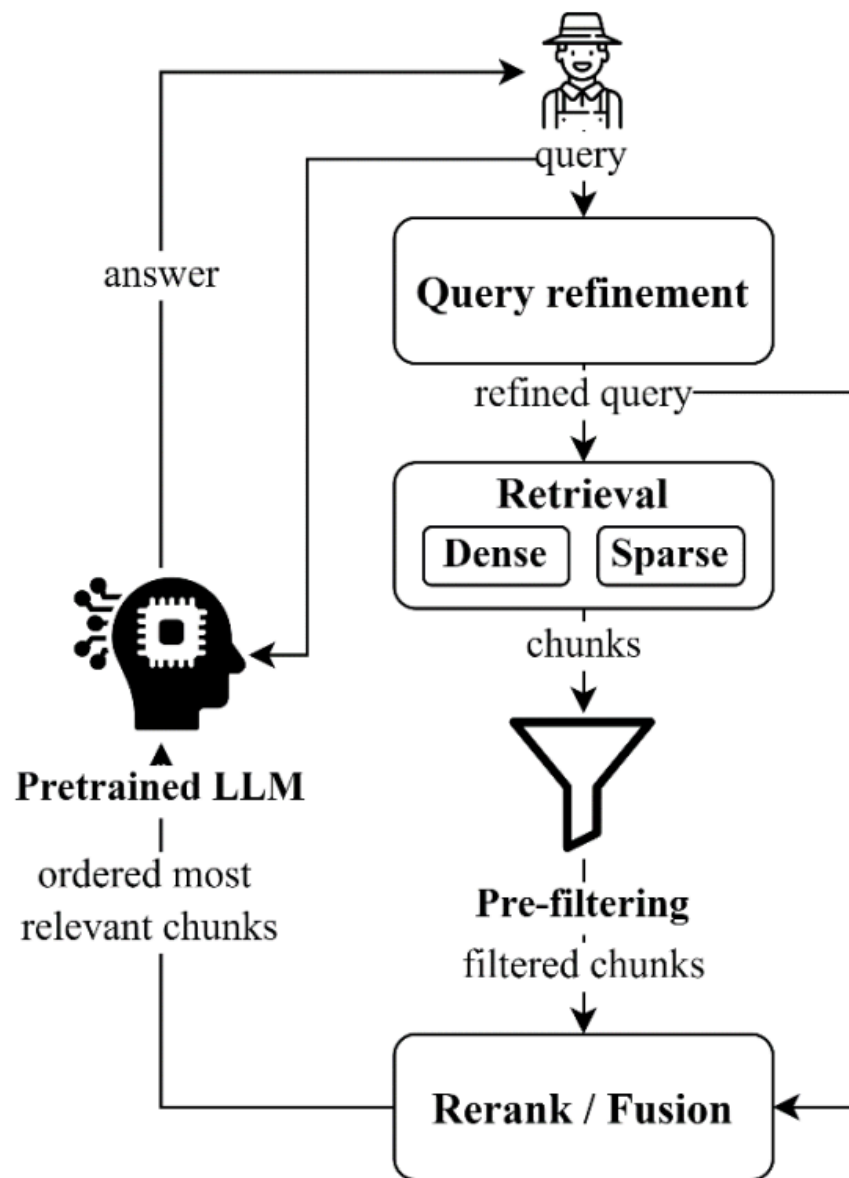
dengan $\mathcal{R}$ sebagai komponen *retriever*, $\mathcal{D}$ sebagai sumber pengetahuan, $\mathcal{G}$ sebagai komponen *generator*, dan a sebagai jawaban yang dihasilkan berdasarkan kueri q serta konteks yang diperoleh dari proses pengambilan informasi.

Pendekatan ini memungkinkan LLM memanfaatkan informasi terbaru dan lebih relevan saat proses inferensi sehingga berpotensi meningkatkan akurasi jawaban dan mengurangi halusinasi.

II.3.2 Komponen Utama RAG

Vizniuk **and others** (december 2024) menjelaskan bahwa keterbatasan basis pengetahuan internal *Large Language Model* (LLM) yang statis dan rentan terhadap informasi usang dapat diatasi secara dinamis melalui arsitektur *Retrieval-Augmented Generation* (RAG). Kerangka kerja RAG secara garis besar terdiri atas tiga komponen utama, yaitu pengambilan informasi (*retrieval*) dari sumber eksternal, augmentasi prompt (*augmentation*), dan generasi respons (*generation*). Alur kerja atau *inference pipeline* dari sistem RAG ini dimulai dari masukan kueri pengguna hingga dihasilkan jawabannya oleh model seperti yang diilustrasikan pada Gambar ??.

Menurut Kamalipour, Asadi **and** Chimeh (2026), sistem RAG terdiri atas empat

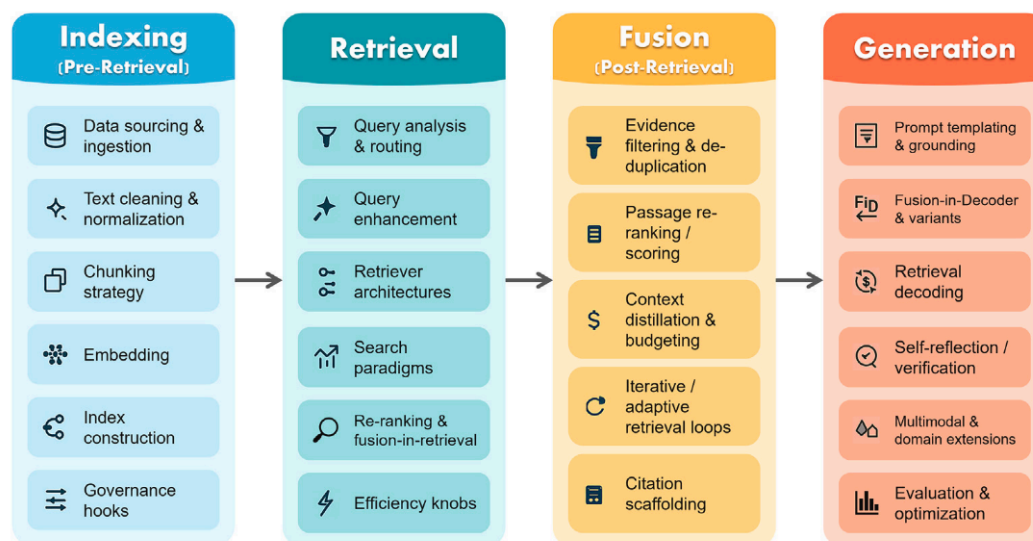


Gambar II.3 *Inference Pipeline* RAG modular berdasarkan taksonomi Vizniuk and others (december 2024).

tahap utama, yaitu *Indexing*, *Retrieval*, *Fusion*, dan *Generation*. Semua tahapan ini saling berhubungan, sebagaimana ditunjukkan pada Gambar ??.

II.3.2.1 Pengindeksan Basis Pengetahuan (*Indexing*).

Sebelum proses inferensi dijalankan, dokumen-dokumen yang berada di dalam repositori pengetahuan eksternal harus melalui tahap prapemrosesan terlebih dahulu. Dokumen berukuran besar dipecah menjadi segmen-segmen tekstual yang lebih kecil dan padat yang disebut *chunks*. Setiap *chunk* teks kemudian dikonversi menjadi



Gambar II.4 Taksonomi RAG yang terdiri atas tahap *Indexing*, *Retrieval*, *Fusion*, dan *Generation* Kamalipour, Asadi and Chimeh (2026)

representasi vektor numerik tingkat tinggi (vektor *embedding*) menggunakan model *encoder* (umumnya berbasis arsitektur *Encoder-only* seperti BERT). Seluruh representasi vektor ini disimpan ke dalam basis data vektor (*vector database*) untuk memfasilitasi pencarian semantik yang cepat dan efisien.

II.3.2.2 Penyempurnaan Kueri (*Query Refinement*).

Tahap awal dalam alur inferensi dimulai ketika pengguna memasukkan kueri tekstual. Mengingat kueri mentah dari pengguna sering kali mengandung ambiguitas atau kesalahan ketik, sistem dapat menerapkan prosedur penyempurnaan kueri (*query refinement*) secara opsional sebelum pencarian dilakukan. Langkah ini bertujuan meningkatkan akurasi pengambilan informasi melalui koreksi kesalahan tata bahasa, ekspansi kueri (*query expansion*) menggunakan istilah-istilah terkait, ataupun memperjelas frasa yang samar agar maksud pencarian lebih eksplisit.

II.3.2.3 Pengambilan Informasi (*Retrieval*).

Kueri yang telah disempurnakan kemudian diubah menjadi vektor kueri oleh model *encoder* yang sama. Sistem selanjutnya mengeksekusi pencarian untuk mengidentifikasi sejumlah *chunks* dokumen yang paling relevan dengan kueri. Vizniuk and others (december 2024) menekankan bahwa implementasi modern kerap memanfaatkan pendekatan *hybrid retrieval* yang menggabungkan metode pencarian berbasis term/kata kunci (*sparse retrieval* seperti BM25) dengan pencarian berbasis kedekatan semantik (*dense retrieval*). Salah satu metrik matematis yang umum digunakan untuk mengukur tingkat kemiripan semantik antar-vektor di dalam ruang

multidimensi adalah kemiripan kosinus (*cosine similarity*):

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|} \quad (\text{II.4})$$

dengan $\mathbf{q}$ menyatakan vektor kueri dan $\mathbf{d}$ menyatakan vektor dokumen (*chunk*). Semakin tinggi nilai kosinus yang dihasilkan, semakin dekat pula hubungan semantik antara potongan informasi tersebut dengan pertanyaan pengguna.

II.3.2.4 Prapenyaringan dan Pengurutan Ulang (*Pre-filtering & Reranking*).

Setelah kandidat dokumen terkumpul dari proses pencarian, sistem dapat menjalankan dua langkah optimasi opsional berikut untuk memastikan kualitas informasi:

1. ***Pre-filtering***: Berfungsi untuk mengeliminasi secara langsung *chunks* informasi yang terdeteksi tidak relevan atau memiliki skor kemiripan di bawah ambang batas tertentu sebelum masuk ke tahap pemrosesan berikutnya.
2. ***Rerank / Fusion***: Berperan mengurutkan ulang *chunks* dokumen yang lolos saringan demi menempatkan informasi yang paling bernilai tinggi pada prioritas teratas. Pendekatan ini dapat diselesaikan melalui metode *Reciprocal Rank Fusion* (RRF) yang efisien karena hanya mengevaluasi posisi peringkat mentah hasil pencarian, atau menggunakan model *Cross-Encoder* yang menghitung ulang skor kemiripan kueri-dokumen secara mendalam (lebih presisi namun membutuhkan beban komputasi yang lebih besar).

II.3.2.5 Augmentasi Konteks dan Generasi respons (*Augmentation & Generation*).

Tahap akhir dari *pipeline* ini adalah proses sintesis jawaban. Sejumlah potongan teks eksternal yang menempati peringkat teratas digabungkan ke dalam satu templat instruksi bersama dengan kueri asli pengguna untuk membentuk sebuah *augmented prompt*. Prompt yang kaya akan konteks faktual ini kemudian dikirimkan ke *Large Language Model* (biasanya berbasis arsitektur *Decoder-only* atau *Encoder-Decoder*). Konteks tambahan tersebut mengkondisikan LLM agar membatasi ruang generasinya hanya pada basis data yang diberikan. Melalui mekanisme augmentasi ini, LLM mampu menghasilkan jawaban yang akurat, sesuai konteks domain spesifik, serta secara signifikan meminimalkan risiko terjadinya halusinasi informasi (*hallucination*).

II.3.3 Keterbatasan RAG Konvensional dalam Domain Pemrograman

Meskipun RAG telah menunjukkan kinerja yang baik pada berbagai bidang, implementasi RAG konvensional masih memiliki beberapa keterbatasan, khususnya pada domain pemrograman. Menurut Huang **and** Huang (2026), pendekatan *retrieval* berbasis kemiripan vektor cenderung berfokus pada kesamaan teks, tetapi belum mampu menangkap hubungan semantik yang kompleks antar konsep.

Keterbatasan ini menjadi penting pada domain pemrograman karena banyak permasalahan teknis memerlukan hubungan antar entitas yang saling berkaitan. Sebagai contoh, pertanyaan mengenai penanganan `AttributeError` pada operasi *dataframe* di Pandas tidak hanya membutuhkan informasi mengenai objek `DataFrame`, tetapi juga hubungan dengan metode yang digunakan, penyebab munculnya *exception*, solusi yang telah divalidasi komunitas, serta keterkaitan dengan konsep lain yang relevan.

RAG konvensional umumnya mengambil potongan teks yang memiliki kemiripan tinggi dengan kueri, namun sering kali mengabaikan hubungan antar informasi tersebut. Akibatnya, konteks yang diberikan kepada LLM menjadi kurang lengkap dan dapat memengaruhi kualitas jawaban yang dihasilkan.

Hendawi2026<empty citation> menunjukkan bahwa penambahan konteks yang lebih kaya secara relasional mampu meningkatkan akurasi respons LLM. Temuan ini mengindikasikan bahwa representasi pengetahuan yang tidak hanya berbasis kemiripan teks, tetapi juga mempertimbangkan hubungan antar entitas, menjadi penting terutama untuk permasalahan pemrograman yang memerlukan penalaran bertahap dan keterkaitan antarkonsep.

II.4 Knowledge Graph

II.5 Knowledge Graph

II.5.1 Definisi dan Representasi Formal

Knowledge graph (KG) adalah bentuk representasi pengetahuan berbasis graf yang digunakan untuk memodelkan entitas dan hubungan antar entitas dalam suatu domain (Hogan **and** others 2022; Pan **and** others 2024). Pada KG, informasi direpresentasikan dalam bentuk simpul (*node*) sebagai entitas dan sisi (*edge*) sebagai hubungan antar entitas.

Secara formal, sebuah KG dapat didefinisikan sebagai kumpulan tripel:

$$G = \{(h, r, t) \mid h, t \in \mathcal{E}, r \in \mathcal{R}\} \quad (\text{II.5})$$

dengan $\mathcal{E}$ sebagai himpunan entitas dan $\mathcal{R}$ sebagai himpunan relasi. Setiap tripel (h, r, t) menunjukkan bahwa entitas *head* (h) memiliki hubungan tertentu (r) dengan entitas *tail* (t).

Dalam penelitian ini yang berfokus pada domain pemrograman, contoh relasi yang dapat dibentuk antara lain:

- (pandas.DataFrame, has_method, groupby())
- (MemoryError, caused_by, large_dataframe_load)
- (chunked_reading, solves, MemoryError)

Relasi-relasi tersebut memungkinkan sistem menyimpan pengetahuan teknis secara lebih terstruktur dibandingkan penyimpanan dokumen biasa.

Pada penelitian ini digunakan *knowledge graph* berbobot (*weighted knowledge graph*), yaitu graf yang menambahkan nilai bobot pada simpul maupun sisi untuk merepresentasikan tingkat kepercayaan informasi. Secara formal dapat dituliskan sebagai:

$$G_w = \{(h, r, t, w_{ht}, w_h, w_t) \mid h, t \in \mathcal{E}, r \in \mathcal{R}, w \in \mathbb{R}^+\} \quad (\text{II.6})$$

dengan w_{ht} menyatakan bobot relasi, sedangkan w_h dan w_t menunjukkan bobot pada simpul. Contoh visual KG berbobot ditunjukkan pada Gambar ??.

Gambar II.5 Contoh *knowledge graph* berbobot pada domain pemrograman. Simpul merepresentasikan entitas teknis, sisi merepresentasikan relasi, dan bobot menunjukkan tingkat kepercayaan informasi.

II.5.2 Konstruksi *Knowledge Graph* dari Teks Tidak Terstruktur

Pembangunan KG dari dokumen teks dilakukan melalui beberapa tahap. Menurut Zhou **and others** (2026), proses konstruksi KG umumnya terdiri atas empat langkah utama, yaitu: (1) penyusunan ontologi domain, (2) praproses dokumen, (3) identifikasi entitas, dan (4) ekstraksi relasi hingga terbentuk tripel pengetahuan.

Alur proses tersebut ditunjukkan pada Gambar ??.

Gambar II.6 Tahapan pembangunan *knowledge graph* dari dokumen sumber, mulai dari pembentukan ontologi hingga ekstraksi entitas dan relasi. Diadaptasi dari Zhou **and others** (2026), Fig. 6.

Tahap identifikasi entitas dilakukan menggunakan metode *Named Entity Recognition* (NER). Proses ini menghasilkan himpunan entitas:

$$\mathcal{E} = \{e_i \mid e_i = (name, type, span)\} \quad (II.7)$$

dengan:

- *name*: nama entitas yang ditemukan
- *type*: kategori entitas
- *span*: posisi entitas pada dokumen

Setelah entitas diperoleh, tahap berikutnya adalah ekstraksi hubungan (*relation extraction*) untuk membentuk tripel:

$$\mathcal{R} = \{t_{ij} \mid t_{ij} = (e_i, r_{ij}, e_j)\} \quad (II.8)$$

dengan r_{ij} menunjukkan hubungan antar pasangan entitas.

Hasil penelitian Zhou **and others** (2026) menunjukkan bahwa kualitas KG sangat memengaruhi performa sistem. Integrasi KG dan LLM menghasilkan akurasi 90,57%, lebih tinggi dibandingkan RAG berbasis dokumen (86,95%) maupun LLM tanpa dukungan pengetahuan eksternal (55,79%). Temuan ini menunjukkan bahwa struktur pengetahuan yang baik dapat meningkatkan kemampuan sistem dalam memahami informasi domain.

Namun demikian, pembangunan KG masih menghadapi beberapa tantangan seperti duplikasi entitas, relasi yang tidak konsisten, dan konflik makna antar tripel. Untuk mengatasi masalah tersebut, Li **and others** (2026) mengusulkan mekanisme perbaikan iteratif menggunakan *competency questions* (CQ) sebagai umpan balik untuk memperbaiki kualitas graf.

II.5.3 Keunggulan Representasi Berbasis Graf dan Integrasi dengan LLM

Menurut Pan **and others** (2024), integrasi antara LLM dan KG membentuk pendekatan baru yang menggabungkan kemampuan penalaran bahasa alami dari LLM dengan pengetahuan terstruktur dari KG. Kombinasi ini membantu mengurangi halusinasi karena model memperoleh informasi yang dapat ditelusuri dan diverifikasi.

Dibandingkan penyimpanan berbasis vektor pada RAG konvensional, representasi berbasis graf memiliki beberapa keunggulan penting, terutama untuk domain pem-

rograman.

Pertama, mendukung penalaran multi-hop.

KG memungkinkan sistem mengikuti hubungan antar entitas secara bertahap. Sebagai contoh:

```
DataFrame.apply() → row_by_row_processing → high_memory_usage → MemoryError
```

Rantai hubungan tersebut membantu sistem memahami penyebab masalah secara lebih lengkap. Mekanisme seperti ini sulit dicapai hanya menggunakan pencarian berbasis kemiripan vektor.

Kedua, mampu merepresentasikan struktur hierarki domain.

Sebagai contoh:

```
pandas.DataFrame → pandas.NDFrame → numpy.ndarray
```

Hubungan hierarkis tersebut penting untuk memahami keterkaitan antar objek dalam domain pemrograman.

Ketiga, mendukung pemberian bobot kualitas informasi.

Berbeda dengan representasi vektor biasa, KG memungkinkan setiap simpul dan relasi memiliki bobot tertentu yang menunjukkan tingkat kepercayaan atau kepentingan informasi.

Tao **and others** (2026) menunjukkan bahwa penambahan KG pada sistem RAG dapat meningkatkan performa secara bertahap, mulai dari RAG konvensional menuju KG-RAG hingga pendekatan yang menggabungkan struktur relasional dan preferensi semantik. Hasil tersebut menunjukkan bahwa KG menjadi komponen penting untuk meningkatkan keandalan sistem berbasis LLM, terutama pada domain teknis seperti pemrograman.

II.6 Graph-based Retrieval-Augmented Generation

II.6.1 Motivasi dan Definisi

Graph-based Retrieval-Augmented Generation (GraphRAG) berkembang sebagai respons terhadap keterbatasan *Vector RAG* dalam menangani kueri yang memerlukan penalaran relasional dan hubungan antar entitas. Pada RAG konvensional, proses pengambilan informasi umumnya bergantung pada kemiripan vektor sehingga lebih efektif untuk menemukan dokumen yang relevan secara topikal, tetapi masih terbatas dalam menangkap hubungan semantik yang kompleks.

Kamalipour, Asadi **and** Chimeh (2026) mendefinisikan GraphRAG sebagai paradigma yang mengintegrasikan proses *retrieve-then-generate* dengan representasi pengetahuan berbasis graf. Dalam pendekatan ini, simpul (*nodes*) merepresentasikan entitas atau konsep, sedangkan sisi (*edges*) menggambarkan relasi di antara entitas tersebut. Struktur ini memungkinkan proses pengambilan informasi yang lebih presisi, mudah dijelaskan (*explainable*), dan kaya konteks, khususnya untuk pertanyaan yang memerlukan penalaran multi-hop.

Pan **and others** (2024) menjelaskan bahwa integrasi antara LLM dan *knowledge graph* membentuk paradigma baru yang meningkatkan kemampuan penalaran berbasis pengetahuan sekaligus membantu mengurangi halusinasi faktual. Sejalan dengan itu, Zhu **and others** (2026) mengidentifikasi tiga komponen utama GraphRAG, yaitu: (1) representasi pengetahuan berbasis graf, (2) mekanisme *retrieval* yang mendukung penalaran multi-hop, dan (3) integrasi pengetahuan yang mempertahankan struktur relasional.

II.6.2 Arsitektur Pipeline GraphRAG

Kamalipour, Asadi **and** Chimeh (2026) mendeskripsikan GraphRAG sebagai serangkaian proses yang dimulai dari pembangunan *knowledge graph*, pengambilan informasi berbasis graf, hingga generasi jawaban menggunakan LLM. Alur umum tersebut ditunjukkan pada Gambar ??.

Gambar II.7 *Pipeline* GraphRAG mulai dari konstruksi KG hingga proses generasi: ekstraksi triple SPO, penyimpanan ke *graph database*, pemetaan entitas kueri, traversal multi-hop, pembentukan subgraf relevan, penyusunan *structured prompt*, dan generasi jawaban oleh LLM. [Gambar dibuat dengan TikZ]

II.6.2.0.1 Konstruksi Knowledge Graph. Tahap awal dilakukan dengan membagi korpus sumber menjadi beberapa segmen yang lebih kecil dan mudah diproses.

Selanjutnya, LLM atau pendekatan *Open Information Extraction* (OpenIE) digunakan untuk mengekstraksi tripel *subject-predicate-object* (SPO). Tripel-tripel tersebut kemudian ditransformasikan menjadi simpul dan relasi dalam graf, lalu disimpan pada *graph database* seperti Neo4j. Selain itu, representasi vektor (*embedding*) untuk setiap simpul dapat ditambahkan untuk mendukung mekanisme pengambilan hibrid antara graf dan vektor (Kamalipour, Asadi **and** Chimeh 2026).

II.6.2.0.2 Retrieval dan Penalaran Berbasis Graf. Pada tahap ini, sistem memetakan entitas yang muncul pada kueri pengguna ke simpul yang sesuai melalui proses *Named Entity Recognition* (NER) dan *entity linking*. Setelah entitas awal diperoleh, proses pengambilan informasi dilakukan menggunakan tiga mekanisme utama (Kamalipour, Asadi **and** Chimeh 2026).

Pertama, *Multi-hop Traversal*, yaitu penelusuran rantai relasi antar entitas untuk membangun konteks yang lebih lengkap. Kedua, *Subgraph Extraction*, yaitu pengambilan subgraf lokal di sekitar entitas yang relevan. Ketiga, *Community Detection*, yaitu identifikasi kelompok entitas yang saling terhubung untuk mendukung pertanyaan global atau proses ringkasan.

Pendekatan ini memungkinkan sistem menangkap hubungan semantik yang tidak dapat direpresentasikan hanya melalui kemiripan vektor.

II.6.2.0.3 Integrasi Konteks Berbasis Struktur. Subgraf hasil proses *retrieval* selanjutnya dikonversi menjadi representasi tekstual yang tetap mempertahankan hubungan antar entitas. Representasi tersebut disusun menjadi *structured prompt* sebelum dikirimkan ke LLM.

Dengan mempertahankan struktur relasional, LLM memperoleh konteks yang lebih kaya dibandingkan ketika hanya menerima potongan teks tidak terstruktur. Akibatnya, jawaban yang dihasilkan menjadi lebih konsisten, dapat ditelusuri, dan berpotensi mengurangi halusinasi.

II.6.3 Perbandingan Kuantitatif: Vector RAG dan GraphRAG

Perbedaan utama antara Vector RAG dan GraphRAG terletak pada mekanisme pengambilan informasi. Vector RAG mengandalkan pencarian berbasis kemiripan vektor, sedangkan GraphRAG memanfaatkan struktur relasional melalui traversal graf. Ringkasan perbandingan keduanya disajikan pada Tabel ??.

Kamalipour, Asadi **and** Chimeh (2026) menunjukkan bahwa GraphRAG membe-

rikan keunggulan yang lebih nyata pada pertanyaan kompleks yang memerlukan penalaran multi-hop. Pada beberapa eksperimen di domain keuangan, kesehatan, dan hukum, penambahan *knowledge graph* meningkatkan proporsi jawaban benar sekitar 30% dibandingkan pendekatan berbasis pengambilan vektor saja.

Meskipun demikian, GraphRAG juga memiliki keterbatasan. Kualitas hasil sangat bergantung pada akurasi konstruksi graf dan keberhasilan proses *entity linking*. Kesalahan pada tahap ini dapat menyebabkan hilangnya jalur penalaran sehingga informasi penting tidak ikut diambil.

II.6.4 Bukti Empiris Efektivitas GraphRAG

Berbagai penelitian menunjukkan bahwa integrasi struktur graf dapat meningkatkan akurasi sekaligus menurunkan tingkat halusinasi pada sistem berbasis LLM.

II.6.4.1 Pythia-RAG (Ali and others 2026).

Pythia-RAG membangun *Multimodal Knowledge Graph* (MMKG) dari sumber tekstual dan visual, kemudian mengambil subgraf yang relevan menggunakan algoritma *Prize-Collecting Steiner Tree*. Subgraf tersebut diproses lebih lanjut menggunakan Graphormer untuk mempertahankan struktur relasional.

Evaluasi pada dataset ScienceQA dan MultiModalQA menunjukkan peningkatan akurasi relatif masing-masing sebesar 5,4% dan 4,8%. Selain itu, *hallucination rate* berhasil ditekan menjadi 8,3%, lebih rendah dibandingkan GPT-4o (17,6%) dan MuRAG (14,2%). Hasil ini menunjukkan bahwa penggunaan KG sebagai sumber *grounding* mampu mengurangi halusinasi secara signifikan.

II.6.4.2 PRIME (Wang 2026).

PRIME (*Progressive Reasoning via Incremental Multi-hop Exploration*) mengusulkan mekanisme penalaran bertahap berbasis subgraf yang memungkinkan eksplorasi hubungan semantik secara progresif.

Metode ini mencapai performa sebesar 91,6% Hit@1 pada WebQSP dan 78,3% F1 pada CWQ, sekaligus menunjukkan efisiensi komputasi yang tinggi karena tidak memerlukan pelatihan tambahan.

II.6.4.3 LARK (Khalifa, Messaoud and Mkaouer 2026).

LARK (*License Analysis with RAG and Knowledge Graphs*) menerapkan GraphRAG pada analisis lisensi perangkat lunak. Evaluasi terhadap sekitar 4.000 proyek Python sumber terbuka menunjukkan akurasi deteksi lisensi sebesar 98,1% dan F1 kompatibilitas sebesar 96,2%.

Tingkat halusinasi yang diperoleh hanya 2,1%, karena KG diposisikan sebagai lapisan pengetahuan yang bersifat otoritatif dan dapat diaudit, sementara LLM dibatasi pada proses pemahaman kueri dan ekstraksi informasi.

II.6.5 Aplikasi GraphRAG pada Domain Teknis

Berbagai studi menunjukkan bahwa GraphRAG memberikan manfaat terbesar pada domain yang memiliki struktur pengetahuan kuat dan memerlukan penalaran relasional.

Awotunde **and others** (2026) menerapkan pendekatan KG + RAG pada *text-to-SQL generation* untuk basis data spesifik-domain dan menemukan bahwa pendekatan tersebut mampu mengatasi keterbatasan RAG konvensional maupun *fine-tuning*. Sementara itu, Cong **and others** (2026) menunjukkan bahwa struktur relasional dalam KG meningkatkan kualitas pengambilan konteks pada proses ideasi produk.

Karakteristik tersebut sangat mirip dengan domain pemrograman yang menjadi fokus penelitian ini. Pengetahuan pemrograman bersifat terstruktur, melibatkan entitas seperti fungsi, kelas, pustaka, serta relasi seperti *requires*, *raises*, dan *solves*. Selain itu, banyak pertanyaan teknis memerlukan penalaran multi-hop untuk menemukan akar masalah dan solusi yang tepat.

Stack Overflow menyediakan sumber pengetahuan komunitas yang kaya sekaligus memiliki sinyal kualitas berupa skor, jawaban terpilih, dan interaksi pengguna. Informasi ini dapat dimanfaatkan sebagai bobot kepercayaan dalam *knowledge graph*, sehingga mendukung proses pengambilan informasi yang lebih akurat.

II.6.6 Sintesis dan Posisi Penelitian

Berdasarkan kajian literatur, Vector RAG memiliki keunggulan pada skalabilitas dan kemampuan pengambilan informasi berbasis topik. Namun, pendekatan ini masih terbatas dalam menangani hubungan relasional dan penalaran multi-hop.

Sebaliknya, GraphRAG menyediakan struktur pengetahuan yang dapat dijelaskan, mendukung penalaran bertingkat, dan menghasilkan konteks yang lebih kaya. Kelemahannya terletak pada kebutuhan konstruksi KG dan sensitivitas terhadap kesalahan *entity linking*.

Penelitian ini memposisikan diri sebagai pengembangan lebih lanjut dari GraphRAG dengan menambahkan sinyal kepercayaan komunitas Stack Overflow sebagai bobot pada *knowledge graph*. Pendekatan tersebut diharapkan mampu meningkatkan kualitas *retrieval*, memperkuat proses *grounding*, dan mengurangi halusinasi pada LLM

untuk domain pemrograman.

II.7 Stack Overflow sebagai Basis Pengetahuan Tervalidasi Sosial

II.7.1 Struktur dan Mekanisme Kurasi Komunitas

Stack Overflow adalah platform *question and answer* (Q&A) yang berfokus pada pemrograman dan pengembangan perangkat lunak. Per 2024, platform ini menyimpan lebih dari 58 juta pasang pertanyaan dan jawaban (Silva, Samhi **and** Khomh 2025). Yang membedakan Stack Overflow dari repositori teks biasa adalah mekanisme kurasi sosial bawaan yang menghasilkan sinyal kualitas terukur: sistem *upvote/downvote*, status *accepted answer*, dan sistem reputasi kontributor.

Silva, Samhi **and** Khomh (2025) merancang studi empiris yang secara eksplisit memanfaatkan tiga mekanisme ini sebagai proksi kualitas jawaban (Gambar ??). Studi tersebut mengonfirmasi bahwa jawaban dengan skor tinggi dan status *accepted* memiliki korelasi positif dengan akurasi teknis yang terverifikasi —menjadikan Stack Overflow sebagai basis pengetahuan tervalidasi secara sosial (*socially-validated knowledge base*).

II.7.2 Sinyal Kepercayaan sebagai Proksi Kualitas

Dari perspektif *knowledge graph* berbobot, sinyal-sinyal kepercayaan komunitas Stack Overflow dapat dioperasionalkan sebagai bobot pada simpul dan sisi graf. Tabel ?? merangkum sinyal yang tersedia beserta interpretasinya.

Tabel II.1 Sinyal kepercayaan komunitas Stack Overflow dan interpretasinya sebagai bobot pada *knowledge graph* berbobot

Sinyal	Sumber	Interpretasi sebagai Bobot
Skor jawaban	Akumulasi <i>upvote</i> – <i>downvote</i>	Bobot simpul jawaban: penerimaan kolektif komunitas
Status <i>accepted answer</i>	Pilihan eksplisit penanya	Bobot biner: validasi langsung oleh praktisi
Reputasi kontributor	Rekam jejak kontribusi historis	Bobot kepercayaan pada sisi kontribusi
Skor pertanyaan	Akumulasi <i>upvote</i> pada pertanyaan	Bobot simpul pertanyaan: relevansi masalah
Jumlah tampilan (<i>view count</i>)	Aktivitas baca pengguna	Bobot popularitas: relevansi jangka panjang

Gambar ?? memetakan sinyal-sinyal tersebut ke dalam elemen-elemen *knowledge graph* berbobot yang akan dibangun dalam penelitian ini.

Gambar II.8 Pemetaan sinyal kepercayaan komunitas Stack Overflow (kiri) ke atribut bobot pada simpul dan sisi *knowledge graph* berbobot (kanan).

II.7.3 Pemanfaatan Konten Stack Overflow dalam Sistem AI

Nashid, Sintaha **and** Mesbah (2023) menunjukkan bahwa konten Stack Overflow sebagai sumber pengambilan konteks meningkatkan kualitas jawaban LLM secara signifikan pada tugas pemrograman. Namun, pendekatan ini menggunakan RAG konvensional berbasis kemiripan vektor—tidak mengeksplorasi struktur relasional maupun sinyal kepercayaan komunitas. Gambar ?? menampilkan distribusi domain pertanyaan yang menghasilkan kemiripan tinggi dan rendah antara jawaban LLM dan komunitas (Silva, Samhi **and** Khomh 2025), mengindikasikan bahwa pola halusinasi LLM tidak terdistribusi merata antar domain—informasi ini penting untuk mengkalibrasi ekspektasi pada evaluasi.

Gambar II.9 Distribusi domain pertanyaan untuk jawaban LLM dengan kemiripan tinggi dan rendah terhadap komunitas Stack Overflow. Diadaptasi dari Silva, Samhi **and** Khomh (2025), Fig. 5.

Kabir **and others** (2024) melengkapi ini dengan studi empiris karakteristik jawaban ChatGPT terhadap pertanyaan Stack Overflow, mengonfirmasi ketidakakuratan faktual yang cukup signifikan untuk menjadi perhatian serius dalam konteks adopsi LLM.

II.8 Evaluasi Keandalan *Large Language Model*

II.8.1 Tantangan Pengukuran Halusinasi

Gu, Jiang, Shi **and others** (2026) menegaskan bahwa mengukur keandalan LLM membutuhkan desain dan standardisasi yang cermat. Evaluasi manual berbasis anotasi manusia terbatas dalam hal skalabilitas. Evaluasi berbasis *benchmark* lebih skalabel, tetapi rentan terhadap kontaminasi data apabila *benchmark* yang digunakan muncul dalam data pelatihan model.

II.8.2 Metrik Evaluasi Halusinasi

Beberapa metrik kuantitatif yang relevan untuk penelitian ini adalah sebagai berikut.

II.8.2.1 Akurasi Faktual (*Factual Accuracy*).

Proporsi klaim faktual dalam jawaban yang dapat diverifikasi sebagai benar terhadap sumber referensi. Dalam domain pemrograman, sumber referensi dapat berupa dokumentasi resmi, kode yang dapat dieksekusi, atau jawaban komunitas SO berskor tinggi.

II.8.2.2 Tingkat Halusinasi (*Hallucination Rate*).

Proporsi jawaban yang mengandung setidaknya satu klaim halusinasi, diukur pada dua level: per-jawaban dan per-klaim (Ji **and others** 2023; Huang **and others** 2025).

II.8.2.3 Skor Konsistensi (*Consistency Score*).

Stabilitas jawaban model terhadap pertanyaan yang semantis serupa namun diformulasikan berbeda (Chang **and others** 2026). Relevan karena Jiang2026<empty citation> mendokumentasikan bahwa *prompt sensitivity*—ketidakstabilan respons LLM terhadap perubahan kecil formulasi—adalah masalah utama dalam aplikasi pemrograman.

II.8.3 Paradigma *LLM-as-a-Judge*

Gu, Jiang, Shi **and others** (2026) mengembangkan kerangka komprehensif untuk paradigma *LLM-as-a-Judge*, di mana LLM yang kuat digunakan sebagai penilai otomatis untuk mengevaluasi keluaran LLM lain. Kerangka ini, yang diilustrasikan pada Gambar ??, mencakup empat tahap evaluasi: (1) masukan konteks dan instruksi evaluasi, (2) inferensi model penilai, (3) *post-processing* skor, dan (4) agregasi untuk reliabilitas.

Gambar II.10 Kerangka keseluruhan survei *LLM-as-a-Judge*: dari definisi formal, klasifikasi, strategi peningkatan reliabilitas, hingga evaluasi dan aplikasi. Diadaptasi dari Gu, Jiang, Shi **and others** (2026), Fig. 1.

Gambar II.11 (A) *Pipeline* evaluasi *LLM-as-a-judge* dari awal hingga akhir, mencakup empat tahap. (B) Metode penilaian: *scoring*, *classification*, dan *ranking*. Diadaptasi dari Gu, Jiang, Shi **and others** (2026), Fig. 2.

Gu, Jiang, Shi **and others** (2026) mengusulkan strategi untuk meningkatkan reliabilitas paradigma ini, termasuk penggunaan rubrik evaluasi terstruktur, teknik *Chain-of-Thought* untuk transparansi penalaran, dan *ensemble* beberapa model penilai. Kerangka ini relevan sebagai protokol evaluasi dalam penelitian ini, di mana penilaian akurasi faktual pada skala besar tidak praktis dilakukan secara manual.

II.9 Sintesis Penelitian Terkait dan *Research Gap*

II.9.1 Analisis Kritis Penelitian Terkait

Tinjauan terhadap penelitian terdahulu mengungkapkan empat kluster aktif yang masing-masing hanya menjawab sebagian dari permasalahan yang diidentifikasi.

II.9.1.0.1 Kluster 1: Evaluasi Komparatif LLM vs Komunitas. Silva, Samhi **and** Khomh (2025) dan Kabir **and others** (2024) menetapkan *baseline* kuantitatif

tentang keterbatasan LLM dibandingkan keahlian komunitas Stack Overflow. Nilai studi ini terletak pada penetapan *ground truth* empiris, namun orientasinya yang sepenuhnya evaluatif tidak mengusulkan mekanisme arsitektural untuk menjembatani kesenjangan.

II.9.1.0.2 Kluster 2: LLM untuk Kualitas Kode. Alomari **and others** (2026) dan Jiang2026<empty citation> mengkonfirmasi keterbatasan LLM dalam tugas pemrograman (refaktorisasi, deteksi duplikasi kode). Fokusnya pada teknik *prompting* dan *fine-tuning*—tidak membahas GraphRAG sebagai strategi *grounding*.

II.9.1.0.3 Kluster 3: RAG Berbasis Konten Stack Overflow. Nashid, Sintaha **and** Mesbah (2023) membuktikan nilai augmentatif konten Stack Overflow tetapi menggunakan *dense retrieval* murni, sehingga tidak mengeksplorasi dimensi relasional dan sinyal kepercayaan komunitas.

II.9.1.0.4 Kluster 4: GraphRAG pada Domain Generik. Zhu **and others** (2026) dan Pan **and others** (2024) membuktikan superioritas GraphRAG dibandingkan RAG konvensional, tetapi menggunakan *knowledge graph* dari korpus generik tanpa sinyal kualitas sosial.

II.9.2 Identifikasi Kesenjangan

Sintesis keempat kluster mengungkapkan tiga kesenjangan metodologis:

G1 — Tidak ada mekanisme *grounding* arsitektural berbasis komunitas SO: studi evaluatif (Silva, Samhi **and** Khomh 2025; Kabir **and others** 2024) tidak mengusulkan arsitektur solusi.

G2 — RAG berbasis SO tidak memanfaatkan dimensi relasional dan sinyal kepercayaan: Nashid, Sintaha **and** Mesbah (2023) menggunakan *dense retrieval* dan mengabaikan *upvote*, *accepted answer*, serta reputasi kontributor.

G3 — GraphRAG belum diaplikasikan pada KG dari komunitas teknis tervalidasi sosial: studi Zhu **and others** (2026) dan Pan **and others** (2024) menggunakan sumber pengetahuan generik tanpa sinyal kepercayaan komunitas yang terstruktur.

Ketiga kesenjangan ini secara kolektif memotivasi pengembangan *GraphRAG Framework* yang secara sistematis mengintegrasikan KG berbobot dari Stack Overflow ke dalam *pipeline* generasi jawaban LLM.

II.9.3 Kerangka Konseptual Penelitian

Berdasarkan sintesis literatur, kerangka konseptual penelitian ini (Gambar ??) menghubungkan komponen-komponen teoritis ke dalam empat jalur yang kohesif:

Gambar II.12 Kerangka konseptual penelitian: integrasi Stack Overflow sebagai sumber pengetahuan tervalidasi sosial, *knowledge graph* berbobot sebagai struktur representasi, mekanisme *graph-guided retrieval*, *grounded generation*, dan protokol evaluasi halusinasi.

Jalur 1 — Konstruksi Pengetahuan. Konten Stack Overflow diekstraksi dan diproses melalui *pipeline information extraction* untuk menghasilkan KG berarah berbobot. Sinyal kepercayaan komunitas (*upvote*, *accepted answer*, reputasi) diinkorporasi sebagai bobot eksplisit pada simpul dan sisi.

Jalur 2 — Pengambilan Konteks. Kueri pengguna digunakan sebagai titik masuk traversal graf. Mekanisme *graph-guided retrieval* mengikuti jalur berbobot untuk mengekstraksi subgraf yang kaya secara semantik dan relasional.

Jalur 3 — Generasi Berbasis Graf. Subgraf hasil ekstraksi dikonversi menjadi *structured prompt* yang dikirimkan ke LLM. *Grounding* berbasis KG menghasilkan jawaban dengan akurasi faktual yang lebih tinggi dibandingkan generasi parametrik murni.

Jalur 4 — Evaluasi dan Validasi. Efektivitas reduksi halusinasi diukur pada tiga kondisi perbandingan (LLM murni, RAG konvensional, GraphRAG) menggunakan protokol evaluasi berbasis paradigma *LLM-as-a-Judge* (Gu, Jiang, Shi and others 2026).

Konsistensi antara masalah yang diidentifikasi, teori yang mendasari, dan artefak yang dirancang merupakan prasyarat kualitas metodologis dalam penelitian desain (Peppers and others 2007).

DAFTAR PUSTAKA

- Ali, Zafar, Yi Huang, Asad Khan, Guilin Qi, Yuxin Zhang, Junlan Feng, Chao Deng **and** Pavlos Kefalas. 2026. “Pythia-RAG: Retrieval-Augmented Generation over a Unified Multimodal Knowledge Graph for Enhanced Question Answering”. *Knowledge-Based Systems* 335:115200. <https://doi.org/10.1016/j.knosys.2025.115200>.
- Alomari, Nawaf, Moussa Redah, Ahmad Ashraf **and** Mohammad Alshayeb. 2026. “Using LLMs to Enhance Code Quality: A Systematic Literature Review”. *Information and Software Technology* 190:107960. <https://doi.org/10.1016/j.infsof.2025.107960>.
- Awotunde, Ifeoluwa, Dharmendra Reddy Chitte, Yongwei Shan, Weihua Sheng **and** Hossein Khaleghian. 2026. “Domain-Specific SQL Generation with LLMs: A Hybrid Framework Combining Knowledge Graphs and Retrieval-Augmentation”. *Advanced Engineering Informatics* 73:104560. <https://doi.org/10.1016/j.aei.2026.104560>.
- Chang, Zongyu, Feihong Lu, Ziqin Zhu, Qian Li, Cheng Ji **and others**. 2026. “A Survey of LLM Alignment: Instruction Understanding, Intention Reasoning, and Reliable Dialogue Generation”. *Neurocomputing* 687:133629. <https://doi.org/10.1016/j.neucom.2025.133629>.
- Cong, Yangfan, Suihuai Yu, Jianjie Chu, Pavan Tejaswi Velivela, Pengchao Wang, Yaoyao Fiona Zhao **and** Stephen Jia Wang. 2026. “A Retrieval-Augmented Method for Explainable Product Ideation: Unifying Conceptual Design Knowledge Graph and Large Language Models”. *Advanced Engineering Informatics* 74:104770. <https://doi.org/10.1016/j.aei.2026.104770>.
- Cruz Cabello, Miguel De la, Tiago Prince Sales **and** Marcos R. Machado. 2025. “AIOps for Log Anomaly Detection in the Era of LLMs: A Systematic Literature Review”. *Intelligent Systems with Applications* 28:200608. <https://doi.org/10.1016/j.iswa.2025.200608>.
- Gu, Jiawei, Xuhui Jiang, Zhichao Shi **and others**. 2026. “A Survey on LLM-as-a-Judge”. *The Innovation* 7:101253. <https://doi.org/10.1016/j.xinn.2025.101253>.

- Haque, Md Asraful. 2025. "LLMs: A Game-Changer for Software Engineers?" *Benchmark Council Transactions on Benchmarks, Standards and Evaluations* 5 (1): 100204. <https://doi.org/10.1016/j.tbench.2025.100204>.
- Hogan, Aidan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier **and others**. 2022. "Knowledge Graphs". *ACM Computing Surveys* 54 (4): 1–37. <https://doi.org/10.1145/3447772>.
- Huang, Lei, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng **and others**. 2025. "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions". *ACM Transactions on Information Systems* 43 (2): 42:1–42:55. <https://doi.org/10.1145/3703155>.
- Huang, Yizheng **and** Jimmy Xiangji Huang. 2026. "A Survey on Retrieval-Augmented Text Generation for Large Language Models". *ACM Computing Surveys* 58 (12): 300:1–300:38. <https://doi.org/10.1145/3805774>.
- Ji, Ziwei, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu **and others**. 2023. "Survey of Hallucination in Natural Language Generation". *ACM Computing Surveys* 55 (12): 1–38. <https://doi.org/10.1145/3571730>.
- Kabir, Samia, David N. Udo-Imeh, Bonan Kou **and** Tianyi Zhang. 2024. "Is Stack Overflow Obsolete? An Empirical Study of ChatGPT Answers to Stack Overflow Questions". *in CHI Conference on Human Factors in Computing Systems*, 1–17. <https://doi.org/10.1145/3613904.3642596>.
- Kamalipour, Amir Abbas, Shahrokh Asadi **and** Mohammad Mahyar Amiri Chimeh. 2026. "From Vectors to Knowledge Graphs: A Comprehensive Analysis of Modern Retrieval-Augmented Generation Architectures". *Computer Science Review* 61:100925. <https://doi.org/10.1016/j.cosrev.2026.100925>.
- Khalifa, Ilyes Ben, Montassar Ben Messaoud **and** Mohamed Wiem Mkaouer. 2026. "LARK: License Analysis with Retrieval-Augmented Generation and Knowledge Graphs". *Information and Software Technology* 197:108164. <https://doi.org/10.1016/j.infsof.2026.108164>.
- Kumar, Pranjal. 2024. "Large Language Models (LLMs): Survey, Technical Frameworks, and Future Challenges". *Artificial Intelligence Review* 57. <https://doi.org/10.1007/s10462-024-10888-y>.

- Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler **and others**. 2020. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. *in Advances in Neural Information Processing Systems*, 33:9459–9474. <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- Li, Ruihao, Chong Chen, Ying Liu, Tao Wang, Haidong Shao **and** Lianglun Cheng. 2026. “A Collaborative Approach Based on Large Language Model and Knowledge Graphs for Information Integration Towards Smart Manufacturing”. *Engineering Applications of Artificial Intelligence* 176:114785. <https://doi.org/10.1016/j.engappai.2026.114785>.
- Nashid, Noor, Mifta Sintaha **and** Ali Mesbah. 2023. “Retrieval-Based Prompt Selection for Code-Related Few-Shot Learning”. *in International Conference on Software Engineering*, 2450–2462. <https://doi.org/10.1109/ICSE48619.2023.00205>.
- Pan, Shirui, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang **and** Xindong Wu. 2024. “Unifying Large Language Models and Knowledge Graphs: A Roadmap”. *IEEE Transactions on Knowledge and Data Engineering* 36 (7): 3580–3599. <https://doi.org/10.1109/TKDE.2024.3352100>.
- Peffer, Ken, Tuure Tuunanen, Marcus A. Rothenberger **and** Samir Chatterjee. 2007. “A Design Science Research Methodology for Information Systems Research”. *Journal of Management Information Systems* 24 (3): 45–77. <https://doi.org/10.2753/MIS0742-1222240302>.
- Qin, Libo, Qiguang Chen, Xiachong Feng, Yang Wu, Yongheng Zhang **and others**. 2025. “Large Language Models Meet NLP: A Survey”. *Frontiers of Computer Science*, <https://doi.org/10.1007/s11704-025-50472-3>.
- Ronanki, Krishna, Beatriz Cabrero-Daniel, Tomas Herda, Stefan Sitkovich, Jennifer Horkoff **and** Christian Berger. 2026. “Recommendations for Efficient and Responsible LLM Adoption within Industrial Software Development”. *Information and Software Technology* 196:108171. <https://doi.org/10.1016/j.infsof.2026.108171>.
- Silva, Leuson Da, Jordan Samhi **and** Foutse Khomh. 2025. “LLMs and Stack Overflow Discussions: Reliability, Impact, and Challenges”. *Journal of Systems and Software* 230:112541. <https://doi.org/10.1016/j.jss.2025.112541>.

- Tao, Ye, Yonghao Liu, Xiugong Qin, Hui Li **and** Xiaoni Wang. 2026. “Semantic Perception and Response Generation for Home Scene Based on Knowledge Graphs and Large Language Models”. *Neurocomputing* 695:134003. <https://doi.org/10.1016/j.neucom.2026.134003>.
- Vizniuk, Artem, Grygorii Diachenko, Ivan Laktionov, Agnieszka Siwocha, Min Xiao **and** Jacek Smola. **december** 2024. “A Comprehensive Survey of Retrieval-Augmented Large Language Models for Decision Making in Agriculture: Unsolved Problems and Research Opportunities”. *Journal of Artificial Intelligence and Soft Computing Research* 15 (2): 115–146. ISSN: 24496499. <https://doi.org/10.2478/jaiscr-2025-0007>.
- Zhou, Xingwei, Shijiang Li, Ying Liu, Wanran Yang, Chengyue Xiong **and** Liang Hou. 2026. “LLM-Based Domain-Specific Knowledge Graph Construction and Hybrid Retrieval-Augmented Generation”. *Journal of Manufacturing Systems* 86:1022–1038. <https://doi.org/10.1016/j.jmansys.2026.01.002>.
- Zhu, Zulun, Tiancheng Huang, Kai Wang, Junda Ye, Xinghe Chen **and** Siqiang Luo. 2026. “Graph-Based Approaches and Functionalities in Retrieval-Augmented Generation: A Comprehensive Survey”. *ACM Computing Surveys*, <https://doi.org/10.1145/3795880>.